

SMC IP Library — Low-Level Design Document (Accellera SystemC + TLM-2.0 LT)

Contents

- SMC IP Library — Low-Level Design Document (Accellera SystemC + TLM-2.0 LT)
 - Table of Contents
 - 0. Temporal Decoupling for Simulation Speed-Up
 - 0.1 Why temporal decoupling
 - 0.2 Vocabulary used throughout this document
 - 0.3 Project-wide TD policy
 - 0.4 Standard initiator skeleton
 - 0.5 Per-IP TD rules — buckets
 - 0.6 Recommended defaults
 - 0.7 Pitfalls and how to detect them
 - 0.8 Verification hooks for TD
 - 0.9 How TD interacts with the rest of this document
 - 1. SMC Fabric (Router + Address Remap)
 - Overview
 - TLM-2.0 interface
 - Internal architecture
 - Register interface
 - Integration
 - Modeling notes
 - 2. AXI Filter (Inbound / Outbound)
 - Overview
 - TLM-2.0 interface
 - Register interface (32 B per filter, see `filter_ctrl.rdl`)
 - Modeling notes
 - 3. CPU Cluster (Rocket wrapper, ISS-backed)
 - Overview
 - 3.1 ISS selection — survey & recommendation
 - 3.2 Build & integration — Spike
 - 3.3 TLM-2.0 interface
 - 3.4 Internal architecture
 - 3.5 Per-hart execution loop
 - 3.6 Bus bridge (Spike memory callback → TLM)
 - 3.7 IRQ wiring (where each line comes from)
 - 3.8 CPU Control register interface (BASE + 0x001_0000, 8 KB)
 - 3.9 Test-bench / debug API
 - 3.10 Modeling notes
 - 4. PLIC
 - Overview
 - TLM-2.0 interface
 - Register interface (RISC-V PLIC layout)
 - Modeling notes
 - 5. CLINT

- Overview
- TLM-2.0 interface
- Register interface
- Modeling notes
- 6. Watchdog Timer (WDT)
 - Overview
 - TLM-2.0 interface
 - Register interface (per instance, 1 KB)
- 7. Bus Error Unit (BEU)
 - Overview
 - TLM-2.0 interface
 - Register interface (4 KB each)
 - Modeling notes
- 8. Boot ROM
 - Overview
 - TLM-2.0 interface
 - Modeling notes
- 9. Scratchpad SRAM
 - Overview
 - TLM-2.0 interface
 - Modeling notes
- 10. Reset Unit
 - Overview
 - TLM-2.0 interface
 - Register interface (BASE + 0x000_2000, 2 KB)
 - Modeling notes
- 11. PLL Wrapper
 - Overview
 - TLM-2.0 interface
 - Register interface (subset)
- 12. MISC Wrapper
 - Overview
 - Register interface
- 13. Debug Module (RISC-V DM)
 - Overview
 - TLM-2.0 interface
 - Register interface (subset, RISC-V Debug spec v0.13.2)
- 14. DMA Engine (iDMA-style)
 - Overview
 - TLM-2.0 interface
 - Sub-modules
 - Register interface (subset)
 - Modeling notes
- 15. Memory Zeroer
 - Overview
 - TLM-2.0 interface
 - Register interface
 - Modeling notes

- 16. Mailbox Unit
 - Overview
 - TLM-2.0 interface
 - Per-mailbox register interface (offset within 2 KB)
 - Modeling notes
- 17. System Timer OCTS
 - Overview
 - TLM-2.0 interface
 - Register interface (32 B)
 - Modeling notes
- 18. eFuse
 - Overview
 - TLM-2.0 interface
 - Register interface
 - Modeling notes
- 19. UART 16550
 - Overview
 - TLM-2.0 interface
 - Register interface (DLAB-multiplexed)
 - Modeling notes
- 20. Log Engine
 - Overview
 - TLM-2.0 interface
 - Register interface
 - Modeling notes
- 21. I2C Controller
 - Overview
 - TLM-2.0 interface
 - Register interface (key)
 - Modeling notes
- 22. AVSBus Controller
 - Overview
 - TLM-2.0 interface
 - Register interface
 - Modeling notes
- 23. GPIO Peripheral
 - Overview
 - TLM-2.0 interface
 - Register interface (per pin)
 - Modeling notes
- 24. Telemetry Receiver
 - Overview
 - TLM-2.0 interface
 - Register interface
 - Modeling notes
- 25. PVT Wrapper
 - Overview
 - TLM-2.0 interface

- Register interface (subset)
- Modeling notes
- 26. I3C Controller (`i3ccore_wrap` , ×6)
 - Overview
 - TLM-2.0 interface
 - Internal architecture
 - Register interface
 - Transaction flows (LT modelling)
 - IRQ wiring
 - Integration in `smc_top`
 - Modeling notes
- Top-level Integration: `smc_top`
 - Address binding rules
 - Build & packaging
- Appendix A — ISS Integration Cookbook (Spike → SystemC TLM-2.0)
 - A.0 Reconciliation with the upstream Spike SystemC/TLM2.0 wrapper guide
 - A.1 Spike API surface that matters
 - A.2 The `iss_hart` abstract interface (one source of truth)
 - A.3 Implementing the Spike backend (`iss_backend_spike.cpp`)
 - A.4 Bus bridge: Spike memory ops → TLM `b_transport`
 - A.4.1 Two equivalent ways of plugging the bridge into Spike
 - A.4.2 Endianness, access width and TLM error semantics
 - A.5 DMI fast-path for ROM and Scratchpad SRAM
 - A.5.1 RAM ownership policy
 - A.6 Interrupt aggregator: SMC IPs → Spike CSRs
 - A.7 Replacing Spike's built-in PLIC / CLINT / Debug Module / Boot ROM with the SMC's TLM versions
 - A.7.1 Spike's built-ins vs. the SMC's TLM modules
 - A.7.2 How to suppress Spike's built-ins (production mode)
 - A.7.3 Early-bring-up scaffold mode (`use_spike_builtin_plic_clint = true`)
 - A.8 Firmware loading — FESVR ELF + optional HTIF
 - A.9 Debug / JTAG — bridging Spike's Debug Module spec to our SystemC `debug_module`
 - A.10 Multi-hart, time and quantum keepers
 - A.10.1 Quantum defaults — reconciled with upstream
 - A.10.2 Interaction between WFI policy and the §0 sync rules
 - A.11 Build system / CMake hooks
 - A.11.1 Alternative — Spike's upstream `--with-systemc` build path
 - A.11.2 Outstanding Spike-side fixes the wrapper depends on
 - A.12 Co-simulation with Dromajo / Rocket Verilator (commit-log diff)
 - A.13 Worked example — booting SMC firmware under the wrapper
 - A.14 First-implementation milestone (mapped from the upstream wrapper guide)
 - A.15 Risks and required Spike upstream changes

SMC IP Library — Low-Level Design Document (Accellera SystemC + TLM-2.0 LT)

This document specifies the per-IP low-level design, integration interfaces, and register interfaces for every IP enumerated in `01_SMC_Architecture.md`. All models are **loosely-timed**: register accesses use `b_transport`, optional DMI is supported for memory-like IPs (ROM, SRAM), and time annotations are coarse approximations.

Convention used throughout this document

- **Initiator socket:** `tlm_utils::simple_initiator_socket<MyMod, 64>` (data width 64-bit by default).
- **Target socket:** `tlm_utils::simple_target_socket<MyMod, 64>`.
- **Register access width:** as defined per IP (32-bit AXI4-Lite or 64-bit AXI4 / mailbox).
- **Reset:** a single boolean `sc_in<bool> rst_n_i;` per IP, sampled on level change inside `SC_METHOD(reset_proc)`.
- **Interrupt:** `sc_out<bool> irq_o;` (active-high, level-triggered unless stated otherwise).
- **Common GP extension:** `smc_axi_extension` (see §6.1 of `01_SMC_Architecture.md`).
- **Quantum keeper:** required for any IP that initiates traffic (CPU, DMA, Zeroer, Log Engine). See §0 Temporal Decoupling for Simulation Speed-Up for the project-wide TD policy, defaults and per-IP rules.

Table of Contents

0. Temporal Decoupling for Simulation Speed-Up
1. SMC Fabric (router + remap)
2. AXI Filter (inbound / outbound)
3. CPU Cluster (Rocket wrapper, ISS)
4. PLIC
5. CLINT
6. Watchdog Timer (WDT)
7. Bus Error Unit (BEU)
8. Boot ROM
9. Scratchpad SRAM
10. Reset Unit
11. PLL Wrapper
12. MISC Wrapper
13. Debug Module
14. DMA Engine (iDMA-style)
15. Memory Zeroer
16. Mailbox Unit
17. System Timer OCTS
18. eFuse
19. UART 16550
20. Log Engine
21. I2C Controller
22. AVSBus Controller
23. GPIO Peripheral
24. Telemetry Receiver

25. PVT Wrapper

26. I3C Controller (`i3ccore_wrap` , ×6)

Each section includes:

- **Overview** — purpose of the IP.
- **TLM-2.0 interface** — sockets, signals, parameters.
- **Internal architecture** — sub-blocks / state machines / data structures.
- **Register interface** — primary register map and access semantics.
- **Integration** — how it connects in `smc_top`.
- **Modeling notes** — LT abstractions and simplifications.

0. Temporal Decoupling for Simulation Speed-Up

Loosely-timed simulation in SystemC/TLM-2.0 only delivers its full speed-up when **temporal decoupling (TD)** is used. This section is the single source of truth for *how* every SMC IP in this document participates in TD, what the project-wide tunables are, and what the synchronization rules are. Every IP section below assumes the conventions defined here.

0.1 Why temporal decoupling

SystemC's scheduler is single-threaded and pays a context-switch (and event-set rebuild) cost every time a process yields. In an LT model where the CPU executes a billion register-only operations between two interrupts, yielding once per instruction makes the simulator I/O-bound on the kernel rather than on instruction work. Temporal decoupling lets each initiator run *ahead* of the global SystemC time `sc_time_stamp()` for some bounded interval — the **quantum** — and only periodically *synchronize* by yielding back to the scheduler. In practice this gives 10×–100× speed-up for CPU-bound and DMA-bound workloads versus a per-cycle / per-instruction yield, while still preserving functional correctness (because all observable side effects still pass through `b_transport` with annotated `sc_time` deltas).

0.2 Vocabulary used throughout this document

Term	Definition
Local time	<code>sc_time_stamp() + qk.get_local_time()</code> — the current "private" time of an initiator that is running ahead of the kernel.
Global quantum	The simulator-wide upper bound on how far any local clock may drift from <code>sc_time_stamp()</code> . Set once at startup via <code>tlm::tlm_global_quantum::instance().set(...)</code> . Default in this project: 1 µs .
Local quantum	A per-IP override. An IP may pick a smaller quantum (e.g. for fine-grained debug) by calling <code>qk.set_global_quantum(my_q)</code> on its private keeper. It must never pick a larger one.
Sync point	A call to <code>qk.sync()</code> (which executes <code>wait(qk.get_local_time())</code> and resets the keeper). After a sync, the IP's local time and <code>sc_time_stamp()</code> are equal.
Annotated delay (<code>sc_time delay</code>)	The <code>delay</code> argument of <code>b_transport</code> . The initiator passes its local-time accumulator in; the target may extend it; on return the initiator must <code>qk.set(delay)</code> (or <code>qk.inc(target_delay - in_delay)</code>) and <i>may</i> sync.

0.3 Project-wide TD policy

1. **One global quantum, set at `sc_main` start.** Default = `1 µs` (≈1000 instructions at the SMC's nominal 1 GHz core clock). Override with the env var `SMC_QUANTUM_NS` or the constructor argument `smc_top(..., quantum_ns)`.
2. **Every initiator owns its own `tlm_utils::tlm_quantumkeeper`.** Never share a keeper across `SC_THREAD`s.
3. **All initiators must sync at four well-defined points** (the "**S4**" rule, used as a coding-review checklist):
 - **S1 — Quantum exhausted:** `if (qk.need_sync()) qk.sync();` at the bottom of every per-step loop.
 - **S2 — Before observing an external event:** prior to `wait(some_event)`, call `qk.sync()` so the event timestamp is correct.

- **S3 — Before driving an output that must be visible *now*** (interrupts, GPIO, mailbox doorbells, OCTS pulses, recovery sideband): `qk.sync()` immediately before `signal.write(true)`.
 - **S4 — Before reading a target whose model state depends on another initiator's progress** (e.g. CPU polling `DMA.STATUS.busy`, CPU polling `OCTS.value`): `qk.sync()` before issuing the read.
- Targets must honour the incoming `delay` argument.** Idempotent register reads/writes simply add their own access latency (`delay += reg_access_ns`). Memory-like targets that grant DMI must update `dmi_data.local_time` so the initiator can run further ahead between syncs.
 - DMI is the second-tier speed-up.** Any target that exposes a memory image (Boot ROM, Scratchpad SRAM, Mailbox payload region) must implement `get_direct_mem_ptr()` and `invalidate_direct_mem_ptr()` so the CPU and DMA models can run **inside** their quantum without even calling `b_transport` for cacheable accesses.
 - Interrupts pre-empt the quantum.** A target that asserts `irq_o` does so at `sc_time_stamp()` (post-sync), and the CPU/PLIC SC_THREAD must wake at the next quantum boundary at the latest. Worst-case interrupt latency is therefore `≤ quantum` and is the dominant tuning consideration when picking the global quantum.

0.4 Standard initiator skeleton

All initiator IPs in this document follow this pattern; each per-IP "Modeling notes" sub-section only documents *deviations*.

```
SC_HAS_PROCESS(my_initiator);
my_initiator(sc_module_name n, sc_time quantum = sc_time(1, SC_US))
: sc_module(n) {
    qk_.set_global_quantum(quantum);
    qk_.reset();
    SC_THREAD(run);
}

void run() {
    for (;;) {
        if (!enabled_) { qk_.sync(); wait(enable_ev_); continue; } // S2

        do_one_unit_of_work(); // may call b_transport(...) below

        if (must_signal_now_) { // S3
            qk_.sync();
            irq_o.write(true);
        }

        if (qk_.need_sync()) qk_.sync(); // S1
    }
}

void issue(uint64_t addr, uint8_t *buf, unsigned len, bool is_write) {
    tlm::tlm_generic_payload trans;
    trans.set_command(is_write ? tlm::TLM_WRITE_COMMAND : tlm::TLM_READ_COMMAND);
    trans.set_address(addr); trans.set_data_ptr(buf); trans.set_data_length(len);
    sc_time delay = qk_.get_local_time(); // hand local time to target
    data_socket->b_transport(trans, delay);
    qk_.set(delay); // accept any extension
    if (qk_.need_sync()) qk_.sync(); // S1
}

private:
    tlm_utils::tlm_quantumkeeper qk_;
```

0.5 Per-IP TD rules — buckets

Every IP in this document falls into one of four buckets. The bucket determines *whether* and *how* the IP participates in TD; it is reproduced in each IP's "Modeling notes" only when it deviates.

Bucket	IPs	Owns a quantum keeper?	Sync rule
A. Active initiator (TD producer)	<code>smc_cpu_cluster</code> (§3, one keeper per hart), <code>dma_engine</code> (§14), <code>memory_zeroer</code> (§15), <code>log_engine</code> (§20)	Yes — mandatory.	S1+S2+S3+S4
B. Bus-side initiator with self-paced firmware	<code>i3c_controller_wrap</code> (§26, controller mode), <code>i2c_controller</code> (§21, host mode), <code>avsbus_controller</code> (§22), <code>mailbox_unit</code> (§16, payload DMA path)	Yes , but with quantum capped by the byte-time of the slowest external bus (so I3C won't drift past one byte; I2C won't drift past one bit).	S1 + S3 (on <code>irq_o</code>)
C. Time-anchored target	<code>clint</code> (§5, <code>mtime</code>), <code>system_timer_octs</code> (§17), <code>wdt</code> (§6), <code>pll_wrapper</code> (§11)	No keeper of its own , but reads/writes consult <code>sc_time_stamp() + delay</code> so an initiator polling them inside its quantum sees a coherent virtual time.	Initiator must sync (S4) before polling.
D. Pure target / register-only	<code>plic</code> (§4), <code>bus_error_unit</code> (§7), <code>boot_rom</code> (§8), <code>scratchpad_sram</code> (§9), <code>reset_unit</code> (§10), <code>misc_wrapper</code> (§12), <code>efuse</code> (§18), <code>uart_16550</code> (§19), <code>gpio</code> (§23), <code>telemetry_receiver</code> (§24), <code>pvt_wrapper</code> (§25), <code>axi_filter</code> (§2), <code>smc_fabric</code> (§1)	No. Just adds its access latency to the incoming <code>delay</code> .	None.

Cross-IP examples:

- **CPU → DMA hand-off (Bucket A → Bucket A).** CPU writes `DMA.CTRL.start`. The CPU's keeper *does not* have to sync immediately afterwards — DMA wakes when its own SC_THREAD picks up the descriptor at the next CPU sync. CPU's `qk.sync()` happens naturally on the next S1 (quantum exhausted) or S4 (when firmware polls `DMA.STATUS.busy`).
- **DMA → CPU IRQ (Bucket A → Bucket A).** DMA must apply S3 before `irq_o.write(true)`, otherwise the IRQ is timestamped at DMA's local time, which may be later than `sc_time_stamp()` at the moment the CPU is scheduled, producing a perceived "negative latency" that confuses interrupt-latency tests.
- **CPU polling `OCTS.value` (Bucket A → Bucket C).** OCTS computes its returned counter as `(sc_time_stamp() + delay) / tick_period`. The CPU's S4 sync before polling guarantees the firmware sees a monotonic counter even though the read happens "inside" the quantum.
- **I3C IBI (Bucket B → Bucket A).** When a target injects an IBI in the I3C controller's `flow_active` SC_THREAD, the controller must `qk.sync()` before raising `irq_o`; otherwise the firmware's measured IBI latency is off by up to one byte-time.

0.6 Recommended defaults

Tuned on the OCAH SMC firmware bring-up workload (boot, mailbox handshake, FLR, AVS step). Override per study.

Knob	Default	Where set	Effect
<code>tlm_global_quantum</code>	1 μ s	<code>sc_main</code>	Hard upper bound on every keeper.
<code>cpu.quantum_insts</code>	1000	<code>smc_cpu_cluster</code> ctor	At nominal 1 GHz, equals the global quantum.
<code>cpu.cycle_period</code>	1 ns	<code>smc_cpu_cluster</code> ctor	Used to convert retired instructions back into <code>sc_time</code> .
<code>dma.beat_ns</code>	1 ns	<code>dma_engine</code> ctor	Per-beat AXI delay; sums into the burst's annotated delay.
<code>zeroer.beat_ns</code>	0.5 ns	<code>memory_zeroer</code> ctor	Streaming writes are AW/W bursted; assume best-case.
<code>log_engine.descriptor_ns</code>	20 ns	per-instance	Per-record cost.
<code>i3c.byte_time_pp</code>	80 ns	<code>i3c_controller</code> (12.5 MHz PP)	Caps I3C local time so an IBI is never more than one byte late.
<code>i3c.byte_time_od</code>	25 μ s	<code>i3c_controller</code> (400 kHz OD/I2C-Fast)	Bus-paced; usually sync per byte anyway because <code>byte_time_od > quantum</code> .
<code>mailbox.threshold_ns</code>	0 ns	<code>mailbox_unit</code>	Doorbell IRQs are zero-latency in LT.

Two run modes are supported through a single CMake/CLI switch:

- `SMC_TD_MODE=fast` (default) — quantum = 1 μ s, DMI enabled for ROM/SRAM/Mailbox payload, CPU runs `K=1000` instructions per step. Production performance numbers and firmware regression run here.
- `SMC_TD_MODE=lockstep` — quantum = `SC_ZERO_TIME`, DMI off, CPU `K=1`, all IPs sync after every transaction. Used for ISS↔Verilator co-sim (§A.12) and for any test that asserts cycle-accurate interrupt latency.

0.7 Pitfalls and how to detect them

Symptom	Likely cause	Fix
Interrupt latency test fails by ~1 μ s	Initiator forgot S3 sync before <code>irq_o.write</code>	Add <code>qk_.sync()</code> immediately before the write; covered by the linter rule <code>td-s3-sync-before-irq</code> (see §0.8).
Firmware deadlocks polling <code>STATUS.busy</code> even though "real" status is ready	Polling initiator forgot S4 sync	Add <code>qk_.sync()</code> before the poll loop's <code>b_transport</code> ; or model the poll as <code>wait(status_event)</code> .
Counter (<code>mtime</code> , OCTS, WDT) appears to go backwards in trace	Two initiators read the counter inside overlapping quanta	Force a sync in the time-anchored target's <code>b_transport</code> before it computes its return value.
Simulation slows ~10× when DMA is enabled	Per-beat <code>b_transport</code> is forcing scheduler yields because target also does <code>qk_.sync()</code>	Move the target to Bucket D (pure latency annotation); never sync inside a target.
Determinism breaks across runs	Unrelated initiators sync at different orders	Pin the global quantum, never use wall-clock; ensure all <code>SC_THREAD</code> s are started in a fixed order in <code>smc_top</code> .
DMI accesses skip the fabric's address remap	DMI granted on a region that crosses an Alias / M-mode boundary	Have <code>smc_fabric</code> invalidate its DMI hint after every Alias/M-mode/Xvisor reprogramming write.

0.8 Verification hooks for TD

Each initiator IP exposes:

- `set_quantum(sc_time q)` — runtime override.
- `td_stats_t get_td_stats() const` — `{ syncs, quantum_exhausted, premature_syncs_s2, premature_syncs_s3, premature_syncs_s4, dmi_hits, dmi_misses }`. Used by the test plan to detect both *too few* (correctness risk) and *too many* (perf risk) syncs.
- `dump_td_state(std::ostream&)` — current local time, last-sync time, last `b_transport` delay.
- A static lint rule (`tools/td-lint.py`) walks all `SC_MODULE`s declared as Bucket A/B and checks that:
 1. They declare exactly one `tlm_quantumkeeper qk_` member.
 2. Every `irq_o.write(...)` is preceded within the same basic block by `qk_.sync()`.
 3. Every `b_transport(trans, delay)` call passes `qk_.get_local_time()` as `delay` and follows up with `qk_.set(delay)`.

The unit-test harness asserts `get_td_stats().premature_syncs_* == expected` for canonical workloads so that a regression that inserts a stray `qk_.sync()` (and silently halves performance) is caught at CI time.

0.9 How TD interacts with the rest of this document

- §3 CPU Cluster is the canonical Bucket-A initiator. Per-hart quantum keepers, `quantum_insts_`, and the lock-step / co-sim mode are described in detail in §3.5 and §A.10; the present section is the higher-level policy that those implementations follow.
- §A.10 ("Multi-hart, time and quantum keepers") in the ISS Integration Cookbook gives the concrete Spike-backed implementation; read it together with §0.4 above.
- §A.12 ("Co-simulation with Dromajo / Rocket Verilator") only works in `SMC_TD_MODE=lockstep`; do not attempt cycle-accurate co-sim with the default fast quantum.

- §14 DMA Engine, §15 Memory Zeroer, §20 Log Engine, §21 I2C Controller and §26 I3C Controller are Bucket-B initiators; their per-IP "Modeling notes" only call out *deviations* from §0.4.

1. SMC Fabric (Router + Address Remap)

Overview

Implements the SMC dual-network interconnect: AXI4 high-performance and AXI4-Lite low-performance routing, plus alias remap (8 regions), M-mode remap (8 regions), Xvisor remap (8 regions), and direct-to-NoC outbound path. Implements the local/global aperture model (`LOCAL_BASE` , `GLOBAL_BASE` , `REGION_SIZE`).

TLM-2.0 interface

```
SC_MODULE(smc_fabric) {
    // Inbound initiator-side targets (from external + internal masters)
    tlm_utils::simple_target_socket<smc_fabric> sys_axi_in;
    tlm_utils::simple_target_socket<smc_fabric> sep_axi_in;
    tlm_utils::simple_target_socket<smc_fabric> jtag_axi_in;
    tlm_utils::simple_target_socket<smc_fabric> cpu_in;
    tlm_utils::simple_target_socket<smc_fabric> dma_in;
    tlm_utils::simple_target_socket<smc_fabric> log_in;
    tlm_utils::simple_target_socket<smc_fabric> zeroer_in;

    // Internal target-side initiators (to local resources)
    tlm_utils::simple_initiator_socket<smc_fabric> to_sram;
    tlm_utils::simple_initiator_socket<smc_fabric> to_rom;
    tlm_utils::simple_initiator_socket<smc_fabric> to_plic;
    tlm_utils::simple_initiator_socket<smc_fabric> to_clint;
    tlm_utils::simple_initiator_socket<smc_fabric> to_periph; // peripheral xbar
    tlm_utils::simple_initiator_socket<smc_fabric> to_mailbox;
    tlm_utils::simple_initiator_socket<smc_fabric> to_dmactl;
    tlm_utils::simple_initiator_socket<smc_fabric> to_zeroer_ctl;
    tlm_utils::simple_initiator_socket<smc_fabric> to_fabric_cfg; // self-config regs
    tlm_utils::simple_initiator_socket<smc_fabric> to_filter_in[16];
    tlm_utils::simple_initiator_socket<smc_fabric> to_filter_out[16];

    // Outbound to system NoC
    tlm_utils::simple_initiator_socket<smc_fabric> output_axi;
};
```

Internal architecture

- `address_decoder` — uses `LOCAL_BASE` , `GLOBAL_BASE` , `REGION_SIZE` to classify each transaction as **local** or **remote**.
- `alias_remap` — 8 regions, each `{base, size, offset, cacheable}`. Applies offset and updates `cacheable` flag in `smc_axi_extension`.
- `mmode_remap` / `xvisor_remap` — 8 regions each, selected by `prot[0]` and an internal "current privilege" hint set by the CPU model.
- Source-ID tagger: tags `SMC_ID` for direct path, `MMODE_ID` for M-mode remapped, `OTHER_ID` for Xvisor remapped.
- Pure functional in LT (zero latency); router decisions consume one `SC_METHOD` cycle.

Register interface

Offset (BASE+0x001_2000–0x001_4FFF)	Reg	Notes
0x12000 + N×0x20	ALIAS_REMAP[N]	8 entries: BASE, SIZE, OFFSET, CACHEABLE
0x13000 + N×0x08	MMODE_REMAP[N]	8 entries
0x14000 + N×0x08	XVISOR_REMAP[N]	8 entries
0x10100	LOCAL_BASE	RO = 0xC000_0000
0x10104	GLOBAL_BASE	RW, default 0x4000_0000
0x10108	REGION_SIZE	RW, default 0x0200_0000

Integration

Bound first in `smc_top`. All masters and slaves are stitched here. Filters are instantiated *between* the fabric and the targets they protect.

Modeling notes

- Outstanding transactions, ID remapping and crossbar arbitration **not** modeled in LT (use a comment block for future AT extension).
- Routing decisions cached in an `unordered_map` for fast repeated decode.

2. AXI Filter (Inbound / Outbound)

Overview

Each filter inspects address range, source ID, and AXI `prot` bits, applying allow/deny policy. 16 inbound + 16 outbound instances. AXI4-Lite peripherals additionally use a 3-bit `prot` exact-match filter.

TLM-2.0 interface

```
SC_MODULE(axi_filter) {
    tlm_utils::simple_target_socket<axi_filter>  upstream;
    tlm_utils::simple_initiator_socket<axi_filter> downstream;
    // Error slave is internal (returns DECERR / 0xbadcable read data)
};
```

Register interface (32 B per filter, see `filter_ctrl.rdl`)

Offset	Reg	Description
0x00	ADDR_LO / ADDR_HI	56-bit base address
0x08	MASK_LO / MASK_HI	56-bit mask
0x10	SRCID_VAL	Source ID expected
0x14	SRCID_MASK	Source ID mask
0x18	PROT_CTRL	allow_ns, awprot_req, arprot_req, wfen, rfen
0x1C	STATUS	Per-filter hit/miss counter

Modeling notes

- Implemented as a single class instantiated 32 times.
 - Error response: set `gp.set_response_status(TLM_ADDRESS_ERROR_RESPONSE)` and write `0xBADCAB1E` into the data buffer for reads.
-

3. CPU Cluster (Rocket wrapper, ISS-backed)

Overview

A 1–4 core **RV64GC** model that stands in for the SMC's Rocket cores. Because Rocket itself is published only as Chisel/RTL (`chipsalliance/rocket-chip`), it cannot be linked into a Loosely-Timed SystemC simulation directly. Instead the cluster wraps a **functional, ISA-equivalent ISS** as the per-hart execution engine and surrounds it with TLM-2.0 sockets for memory, MMIO and debug. The result behaves, from the rest of the SMC fabric's point of view, exactly like the Rocket RTL would in a cycle-accurate simulation, but at 10^3 – $10^4\times$ the speed.

Two implementation modes are supported:

- **Stub mode** — a TLM-2.0 traffic driver fed by a CSV/JSON scenario file. Used to bring the SMC fabric up before any firmware exists.
- **ISS-backed mode** (default) — an external RV64GC ISS is linked into the model. This is what runs the real SMC firmware (boot ROM, OCCP loop, AVS, telemetry).

3.1 ISS selection — survey & recommendation

There is no Rocket-specific ISS as such; what is required is a verified RV64GC functional simulator that can be embedded inside SystemC. The community-maintained options that meet the "stable / well-verified / open" bar are:

#	Project	Repository	License	What it gives us	Trade-off
1	Spike (riscv-isa-sim)	https://github.com/riscv-software-src/riscv-isa-sim	BSD-3	Golden RISC-V reference model; M/S/U + H, Sv39/48/57, vector, bit-manip, debug spec; built-in PLIC/CLINT models; passes <code>riscv-tests</code> / <code>riscv-arch-test</code>	C++ library — needs a small TLM wrapper
2	DBT-RISE-RISCV / TGC (MINRES)	https://github.com/Minres/DBT-RISE-RISCV , https://github.com/Minres/TGC-ISS , https://github.com/Minres/SystemC-Components	BSD/BSL	DBT (interpreter + JIT) ISS already wrapped as a SystemC TLM-2.0 initiator with quantum keeper, DMI and CoreDSL-generated decode	Smaller user community; H-extension support is partial — verify if Xvisor is required
3	Dromajo	https://github.com/chipsalliance/dromajo	Apache-2.0	Checkpoint-capable RV64GC, designed for commit-log co-simulation against RTL ; boots Linux on virt-style platform; supports H	Less actively developed than Spike; intended for sign-off rather than primary execution
4	QEMU RISC-V	https://gitlab.com/qemu-project/qemu	GPLv2	Very fast functional emulation	Hard to embed cleanly inside SystemC; license may be problematic in a vendor codebase
5	Sail RISC-V	https://github.com/riscv/sail-riscv	BSD-2	Formal golden model	Slow, used as an oracle, not a runtime ISS
6	RISC-V VP (Uni Bremen)	https://github.com/agra-uni-bremen/riscv-vp	GPL	Full SystemC VP with built-in ISS	GPL licensing
7	Renode	https://github.com/renode/renode	MIT	Whole-system emulator (C#)	Co-sim only over a socket, not native SystemC
8	gem5 RISC-V	https://github.com/gem5/gem5	BSD	Architectural research simulator	Overkill for an LT model

Recommendation for this project:

1. **Primary: Spike** as the per-hart execution engine inside `smc_cpu_cluster`. It is the de-facto golden model, BSD-licensed (drops cleanly into a vendor repo), and matches the SMC's required ISA exactly (RV64GC + M/S/U + optional H for Xvisor).
2. **Fallback: DBT-RISE-RISCV / TGC** if a turnkey SystemC initiator with no glue code is preferred — useful when bring-up time matters more than getting the canonical reference.
3. **Sign-off: Dromajo + Rocket Verilator** for later RTL co-simulation. Dromajo's commit log can be diffed against the Rocket emulator output to validate the SMC firmware on real RTL.

The rest of this section assumes **Spike** is the chosen ISS; the wrapper is structured so swapping to TGC/Dromajo only changes one source file (`iss_backend_*.cpp`).

3.2 Build & integration — Spike

Spike is consumed as a static library, not as a top-level executable.

```
git clone https://github.com/riscv-software-src/riscv-isa-sim.git external/spike
cd external/spike && mkdir build && cd build
./configure --prefix=$PWD/install --enable-commitlog --with-isa=rv64gc
make -j$(nproc) && make install
# Produces: install/lib/{libriscv,libsoftfloat,libfesvr,libdisasm}.a
#           install/include/{riscv,fesvr,softfloat,disasm}/...
```

CMake fragment (used by the SMC model build):

```
add_library(spike_iss STATIC IMPORTED GLOBAL)
set_target_properties(spike_iss PROPERTIES
    IMPORTED_LOCATION      ${SPIKE_PREFIX}/lib/libriscv.a
    INTERFACE_INCLUDE_DIRECTORIES ${SPIKE_PREFIX}/include)

target_link_libraries(smc_cpu_cluster
    PUBLIC SystemC::systemc
    PRIVATE spike_iss
        ${SPIKE_PREFIX}/lib/libfesvr.a
        ${SPIKE_PREFIX}/lib/libsoftfloat.a
        ${SPIKE_PREFIX}/lib/libdisasm.a)
```

3.3 TLM-2.0 interface

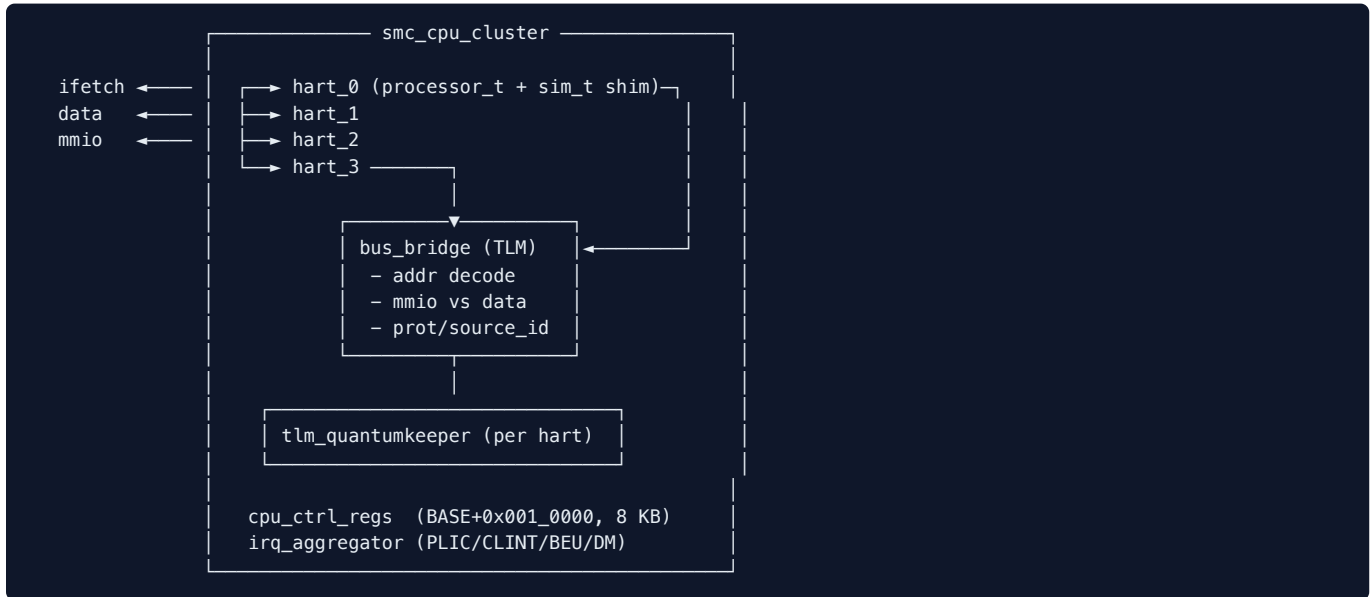
```
SC_MODULE(smc_cpu_cluster) {
    // External buses
    tlm_utils::simple_initiator_socket<smc_cpu_cluster> mmio;    // AXI4-Lite (CSR/peripheral)
    tlm_utils::simple_initiator_socket<smc_cpu_cluster> data;    // AXI4 (memory + DMA-visible)
    tlm_utils::simple_initiator_socket<smc_cpu_cluster> ifetch;  // AXI4 (instruction fetch)
    tlm_utils::simple_target_socket<smc_cpu_cluster> debug_in;  // from debug_module / jtag2axi
    tlm_utils::simple_target_socket<smc_cpu_cluster> ctrl;      // CPU Control regs (back-door)

    // Resets
    sc_in<bool> rst_core_n_i;
    sc_in<bool> rst_uncore_n_i;

    // Interrupts (one bit per hart unless noted)
    sc_vector<sc_in<bool>> meip_in{NCORES}; // M-mode external from PLIC
    sc_vector<sc_in<bool>> seip_in{NCORES}; // S-mode external from PLIC
    sc_vector<sc_in<bool>> mtip_in{NCORES}; // timer from CLINT
    sc_vector<sc_in<bool>> msip_in{NCORES}; // soft from CLINT
    sc_vector<sc_in<bool>> beu_nmi_in{NCORES}; // NMI from BEU
    sc_in<bool> debug_irq_in; // halt request

    // Construction
    smc_cpu_cluster(sc_module_name n,
        unsigned num_harts,
        std::string isa = "rv64gc_zicsr_zifencei",
        std::string priv = "msu");
};
```

3.4 Internal architecture



- One `processor_t` (Spike) per hart, instantiated at construction time.
- Each hart owns an `SC_THREAD` (`hart_thread(id)`) that calls `processor_t::step(K)` in a loop, where `K` is the LT instruction quantum (default 1000 instructions).
- A custom `mmio_device_t`-style **bus bridge** intercepts every Spike load/store and translates it to a TLM `b_transport` on `mmio` / `data` / `ifetch`. The bridge sets the `smc_axi_extension` (`source_id = SMC_ID`, `prot` from current privilege/secure state, `axi_id = hart_id`).
- An **IRQ aggregator** `SC_METHOD` samples `meip/seip/mtip/msip/beu_nmi/debug_irq` and writes Spike's `mip` CSR via `processor_t::set_mip()` between steps.

3.5 Per-hart execution loop

```

void smc_cpu_cluster::hart_thread(unsigned id) {
    auto& proc = *harts[id];
    tlm_utils::tlm_quantumkeeper qk;
    qk.reset();
    while (true) {
        if (!rst_core_n_i.read()) { proc.reset(); wait(rst_core_n_i.posedge_event()); }

        update_mip(id);           // sample IRQ ports, push into Spike CSR
        proc.step(quantum_insts); // execute K instructions; bus callbacks issue b_transport

        // Advance LT time. Use IPC parameter (default 1.0 instr/cycle @ clk_smc).
        qk.inc(sc_time(quantum_insts * cycle_period_, SC_NS));
        if (qk.need_sync()) qk.sync();
    }
}

```

3.6 Bus bridge (Spike memory callback → TLM)

```
bool bus_bridge::mmio_load (reg_t a, size_t len, uint8_t* bytes) {
    return tlm_xfer(tlm::TLM_READ_COMMAND, pick_socket(a), a, len, bytes);
}
bool bus_bridge::mmio_store(reg_t a, size_t len, const uint8_t* bytes) {
    return tlm_xfer(tlm::TLM_WRITE_COMMAND, pick_socket(a), a, len,
        const_cast<uint8_t*>(bytes));
}
```

`pick_socket(addr)` chooses `mmio` for AXI4-Lite peripheral ranges (peripheral xbar, PLIC, CLINT, mailbox, etc.) and `data` for the high-perf network (SRAM, ROM, DMA-visible windows). DMI is requested from ROM/SRAM the first time a fetch hits them and cached per hart for fast re-execution.

3.7 IRQ wiring (where each line comes from)

Spike CSR bit	Driven by	Source IP
<code>mip.MEIP</code>	<code>meip_in[hart]</code>	<code>plic</code> M-mode context output
<code>mip.SEIP</code>	<code>seip_in[hart]</code>	<code>plic</code> S-mode context output
<code>mip.MTIP</code>	<code>mtip_in[hart]</code>	<code>clint.mtip_out</code>
<code>mip.MSIP</code>	<code>msip_in[hart]</code>	<code>clint.msip_out</code>
<code>nmi</code>	<code>beu_nmi_in[hart]</code>	<code>bus_error_unit.nmi_o</code>
Debug halt	<code>debug_irq_in</code>	<code>debug_module</code>

The PLIC and CLINT remain **independent TLM target modules** as specified in §4 and §5 — Spike's built-in PLIC/CLINT are *disabled* and bypassed so that the SMC's real register layout, address map and interrupt routing are exercised by the model.

3.8 CPU Control register interface (BASE + 0x001_0000, 8 KB)

Offset	Reg	Description
0x000	<code>RESET_VECTOR_N</code> (×4)	Per-core reset vector (56-bit)
0x040	<code>CORE_ENABLE</code>	Per-core enable bitmap
0x100	<code>LOCAL_BASE</code> (RO)	0xC000_0000
0x104	<code>GLOBAL_BASE</code>	RW
0x108	<code>REGION_SIZE</code>	RW
0x200	<code>INIT_MEM_DONE</code> (RO)	scratchpad init status
0x204	<code>DISABLE_SRAM_AUTO_INIT</code>	RW
0x208	<code>MEM_REPAIR_STATUS</code>	RO

Writes to `RESET_VECTOR_N[i]` translate into `harts_[i]->set_pc(value)` on the next reset deassertion. `CORE_ENABLE` gates the per-hart `SC_THREAD` (a disabled hart sleeps on an `sc_event`).

3.9 Test-bench / debug API

The wrapper exposes these methods for the test-bench (used by `03_SMC_Test_Plan.md`):


```

void load_image(const std::string& elf, uint64_t base); // calls fesvr ELF loader
void set_pc(unsigned hart, uint64_t pc);
void single_step(unsigned hart, unsigned n = 1);
void inject_nmi(unsigned hart);
auto read_csr(unsigned hart, int csr) -> uint64_t;
auto commit_log() -> std::vector<commit_entry>; // when --enable-commitlog

```

`commit_log()` returns Spike's per-instruction trace, which is what makes Dromajo / Rocket-Verilator co-simulation possible later (diff this against the RTL commit log).

3.10 Modeling notes

- **Quantum:** default 1000 instructions $\approx 1 \mu\text{s}$ @ 1 GHz; tunable via constructor argument.
- **Atomicity:** Spike's AMO/LR-SC sequences are issued as a single TLM transaction with `prot[3]=1` ("locked") so the fabric treats them as exclusive.
- **Privilege & security:** `smc_axi_extension.prot[1] = !(priv == M); prot[2] = (mstatus.MPRV ? mstatus.MPP : priv) != U`. This is what the inbound/outbound filters read.
- **DMI:** ROM and SRAM advertise DMI; instruction fetch is therefore zero-cost after first touch. MMIO never advertises DMI.
- **Multi-hart:** `tlm_quantumkeeper::sync()` is called per hart; `sc_time` is monotonic across harts because all sockets ultimately share the SMC fabric.
- **Hot-swap:** changing the ISS backend (Spike $\rightarrow$ TGC $\rightarrow$ Dromajo) requires only that the new backend implements the `iss_hart` interface (`step`, `reset`, `set_pc`, `set_mip`, `read_csr`, `mem_read`, `mem_write`).
- **Future RTL co-simulation:** when Rocket Verilator becomes available, replace `iss_backend_spike.cpp` with `iss_backend_rocket_vsc.cpp` that drives the Verilated model through the same `iss_hart` interface; the SMC fabric, PLIC, CLINT, etc. are unchanged.

4. PLIC

Overview

Standards-compliant RISC-V Platform-Level Interrupt Controller, 332 sources, multi-context (M-mode + S-mode per core). Address window 4 MB at BASE + 0x400_0000.

TLM-2.0 interface

```

SC_MODULE(plic) {
    tlm_utils::simple_target_socket<plic> reg_socket;
    sc_vector<sc_in<bool>> src_in{332}; // level interrupts
    sc_vector<sc_out<bool>> ctx_out{2*NCORES}; // m+s context per core
};

```

Register interface (RISC-V PLIC layout)

Offset	Reg
0x000000	<code>priority[0..331]</code> (32 bits each, only [2:0] used, 7 levels)
0x001000	<code>pending[0..10]</code> (332 bits)
0x002000 + ctx×0x80	<code>enable[ctx]</code> (332-bit bitmap per context)
0x200000 + ctx×0x1000	<code>threshold[ctx]</code> (32-bit)
0x200004 + ctx×0x1000	<code>claim/complete[ctx]</code> (RW)

Modeling notes

- Use a `priority_queue` keyed on (priority, source_id) per context.
- `claim` returns highest pending source, `complete` clears the EIP.
- Atomic claim is a single `b_transport` from CPU side.

5. CLINT

Overview

Per-core machine timer + software interrupt; 64-bit `mtime` shared across cores. 64 KB at BASE + 0x800_0000.

TLM-2.0 interface

```
SC_MODULE(clint) {
    tlm_utils::simple_target_socket<clint> reg_socket;
    sc_in<bool> tick_in;           // driven by ref clock
    sc_vector<sc_out<bool>> msip_out{NCORES}; // software irq
    sc_vector<sc_out<bool>> mtip_out{NCORES}; // timer irq
};
```

Register interface

Offset	Reg	Width
0x0000 + 4×N	<code>MSIP[N]</code>	32
0x4000 + 8×N	<code>MTIMECMP[N]</code>	64
0xBFF8	<code>MTIME</code>	64

Modeling notes

- `mtime` increments inside `SC_THREAD` driven by `clk_ref_period_ns` parameter.

6. Watchdog Timer (WDT)

Overview

Per-core APB4 watchdog (1 KB each). Two-stage timeout: warning IRQ then reset request to reset_unit.

TLM-2.0 interface

```
SC_MODULE(wdt) {
    tlm_utils::simple_target_socket<wdt> reg_socket;
    sc_out<bool> warning_irq_o;
    sc_out<bool> reset_req_o;
};
```

Register interface (per instance, 1 KB)

Offset	Reg
0x000	CTRL (enable, dbg_disable)
0x004	LOAD (timeout in ref-clock ticks)
0x008	WARN_LOAD
0x00C	KICK (write-only “pet”)
0x010	STATUS
0x014	INT_CLR

7. Bus Error Unit (BEU)

Overview

Per-core AXI transaction monitor. Captures decode/slave/timeout errors, logs address & transaction context. Drives a non-maskable interrupt (NMI) directly into the corresponding CPU.

TLM-2.0 interface

```
SC_MODULE(bus_error_unit) {  
    tlm_utils::simple_target_socket<bus_error_unit> reg_socket;  
    tlm_utils::simple_target_socket<bus_error_unit> monitor_socket; // snoop  
    sc_out<bool> nmi_o;  
};
```

Register interface (4 KB each)

Offset	Reg
0x000	CAUSE (DEC/SLV/TIMEOUT)
0x008	ADDR_LO / ADDR_HI
0x010	TXN_INFO (id, prot, source)
0x018	MASK
0x01C	CLEAR

Modeling notes

- The fabric calls `monitor_socket->b_transport()` with a copy of every failed transaction; the BEU updates its registers and asserts `nmi_o`.

8. Boot ROM

Overview

Read-only memory, up to 64 KB, mapped at `0xC004_0000`. Pre-loaded from a binary image at construction time. Supports endianness flipping (per spec, `rom_flip_endianness_i`).

TLM-2.0 interface

```
SC_MODULE(boot_rom) {
    tlm_utils::simple_target_socket<boot_rom> socket;
    sc_in<bool> flip_endianness_i;
    void load_image(const std::string& path);
    // DMI supported; transport_dbg supported.
};
```

Modeling notes

- Implemented as `std::vector<uint8_t>` with `get_direct_mem_ptr()` returning `dmi_data` with `dmi_allowed`. Optional byte-swap is applied on `b_transport` reads when `flip_endianness_i==true`.

9. Scratchpad SRAM

Overview

1 MiB SRAM in 32 banks, mapped at `0xC006_0000`. Auto-zeroed on cold reset unless `disable_sram_auto_init_i` is asserted; reports completion via `init_mem_done_o`.

TLM-2.0 interface

```
SC_MODULE(scratchpad_sram) {
    tlm_utils::simple_target_socket<scratchpad_sram> socket;
    sc_in<bool> rst_n_i;
    sc_in<bool> disable_auto_init_i;
    sc_out<bool> init_done_o;
};
```

Modeling notes

- DMI enabled.
- On reset rising edge, `SC_THREAD` walks the array clearing it; advances `init_done_o` when complete (modelled at zero LT time but with optional `wait()` for performance studies).

10. Reset Unit

Overview

Generates `rst_primary_*`, `rst_core_*` for SMC/REF/PERIPH clock domains. Implements FLR sequence (pre-reset delay + reset hold), isolation request aggregation (`isolate_req_o[31:0]`), and `skip_mem_repair_o`.

TLM-2.0 interface

```
SC_MODULE(reset_unit) {
    tlm_utils::simple_target_socket<reset_unit> reg_socket;
    sc_in<bool> cold_rst_n_i;
    sc_in<bool> powergood_i;
    sc_in<bool> cfg_flr_pf_active_i;
    sc_vector<sc_in<bool>> isolate_req_pin_i{32};

    sc_out<bool> rst_primary_smc_n_o;
    sc_out<bool> rst_primary_ref_n_o;
    sc_out<bool> rst_primary_periph_n_o;
    sc_out<bool> rst_core_smc_n_o;
    sc_vector<sc_out<bool>> isolate_req_o{32};
    sc_out<bool> skip_mem_repair_o;
};
```

Register interface (BASE + 0x000_2000, 2 KB)

Offset	Reg
0x00	RESET_CTRL
0x04	RESET_STATUS
0x10	ISOLATE_REQ_REG (sw control)
0x14	ISOLATE_REQ_PINEN_REG
0x18	ISOLATE_REQ_SMCEN_REG
0x20	ISOLATE_REQ_FLR_COUNTER_VALUE
0x24	ISOLATE_REQ_FLR_RESET_COUNTER_VALUE
0x40	RESET_HOLD_CFG

Modeling notes

- FLR FSM uses two `sc_event` triggered down-counters in a single `SC_THREAD`.

11. PLL Wrapper

Overview

Provides AXI-Lite control over external PLLs; reports lock status. In LT, the model returns immediate “locked” after a programmable delay. 4 KB at BASE + 0x000_3000.

TLM-2.0 interface

```
SC_MODULE(pll_wrapper) {
    tlm_utils::simple_target_socket<pll_wrapper> reg_socket;
    sc_out<bool> pll_locked_o;
    sc_out<sc_uint<32>> freq_mhz_o; // for monitoring
};
```

Register interface (subset)

Offset	Reg
0x000	<code>PLL_CTRL</code> (enable, ref_select)
0x004	<code>PLL_DIV</code> (M, N, P)
0x008	<code>PLL_STATUS</code> (locked, error)
0x00C	<code>PLL_FREQ_MHZ</code> (RO computed)

12. MISC Wrapper

Overview

Scratch registers, chip configuration, NDM (non-debug-module) reset request. 2 KB at BASE + 0x000_2800.

Register interface

Offset	Reg	Notes
0x000–0x0FC	<code>SCRATCH[0..63]</code>	32-bit RW
0x100	<code>CHIP_ID</code>	RO
0x104	<code>PACKAGE_ID</code>	RO
0x108	<code>CHIP_CFG</code>	RW
0x200	<code>NDM_RESET_REQ</code>	WO (drives PLIC source #267)

13. Debug Module (RISC-V DM)

Overview

RISC-V external debug module with abstract command interface and System Bus Access. Provides JTAG2AXI mastering capability via `jtag_axi_in`.

TLM-2.0 interface

```
SC_MODULE(debug_module) {
    tlm_utils::simple_target_socket<debug_module> reg_socket;    // APB
    tlm_utils::simple_initiator_socket<debug_module> jtag2axi_out; // System Bus Access
    sc_vector<sc_out<bool>> halt_req_o{NCORES};
    sc_vector<sc_in<bool>> halted_i{NCORES};
    sc_out<bool> ndmreset_o;
};
```

Register interface (subset, RISC-V Debug spec v0.13.2)

Offset	Reg
0x10	DMCONTROL
0x11	DMSTATUS
0x12	HARTINFO
0x16	ABSTRACTCS
0x17	COMMAND
0x18	ABSTRACTAUTO
0x38	SBCS
0x39	SBADDRESS0..3
0x3C	SBDATA0..3

14. DMA Engine (iDMA-style)

Overview

PULP iDMA frontend + backend + request manager. AXI4-Lite control (512 B at `0xC003_8000`). AXI4 data master with up to 16 outstanding transactions. Supports 1D/2D linear, repeat, scatter-gather; 56-bit address, 64-bit data.

TLM-2.0 interface

```
SC_MODULE(dma_engine) {
    tlm_utils::simple_target_socket<dma_engine> ctrl_socket;
    tlm_utils::simple_initiator_socket<dma_engine> data_socket;
    sc_out<bool> irq_o;
    sc_in<bool> cg_enable_i;
};
```

Sub-modules

- `dma_frontend` — register file, descriptor parser (linear / 2D / repeat / SG), command FIFO (depth `F2M_FIFO_DEPTH` = 4 default).
- `dma_request_manager` — round-robin arbiter between control interfaces (LT default = passthrough).
- `dma_backend` — emits AXI burst transactions through `data_socket`. In LT mode each burst is one `b_transport` with annotated delay = `beats × beat_ns`.

Register interface (subset)

Offset	Reg
0x000	<code>CFG_LOCK</code>
0x008	<code>SRC_ADDR_LO/HI</code>
0x010	<code>DST_ADDR_LO/HI</code>
0x018	<code>LENGTH_LO/HI</code>
0x020	<code>SRC_STRIDE</code> , <code>DST_STRIDE</code>
0x028	<code>REPEAT_COUNT</code>
0x030	<code>CTRL</code> (start, abort, mode)
0x038	<code>STATUS</code> (busy, last_id, error)
0x040	<code>IRQ_EN</code> / <code>IRQ_STATUS</code>

Modeling notes

- Multi-channel: parameterized via `template <int NCTRL, int NMST>`.
- DMA optimisations (R/AW coupling, page-boundary fragmentation) are abstracted; only error responses, completion IDs and IRQ generation are functionally accurate.
- Temporal decoupling (Bucket A, see \$0):** owns one `tlm_quantumkeeper qk_` shared across the FSM `SC_THREAD`. Per-burst `b_transport` passes `qk_.get_local_time()` as the `delay`, sums in `beats × beat_ns`, then `qk_.set(delay)`. S3 sync mandatory before `irq_o.write(true)` so firmware-measured DMA-completion latency matches the annotated burst time.

15. Memory Zeroer

Overview

Three-state FSM (`IDLE` , `ISSUE_ADDR` , `ISSUE_DATA`). 56-bit AXI master, max burst 255. Registers at `0xC003_8200` (512 B).

TLM-2.0 interface

```
SC_MODULE(memory_zeroer) {
    tlm_utils::simple_target_socket<memory_zeroer> ctrl_socket;
    tlm_utils::simple_initiator_socket<memory_zeroer> data_socket;
    sc_out<bool> irq_o;
    sc_out<bool> busy_o;
};
```

Register interface

Offset	Reg
0x00	<code>DEST_ADDR_LO/HI</code> (64-bit)
0x08	<code>SIZE_LO/HI</code> (64-bit)
0x10	<code>CTRL_STATUS</code> (start, busy, error, abort)
0x18	<code>IRQ_EN</code>

Modeling notes

- LT: a single `SC_THREAD` issues `b_transport` writes of zero-filled buffers (`std::vector<uint8_t>(beats*8, 0)`).
- Outstanding-transaction back-pressure not modelled; counter capped at `MAX_OUTSTANDING=32` .
- **Temporal decoupling (Bucket A, see §0)**: per-burst `delay += beats × beat_ns` , `qk_.set(delay)` after each `b_transport` , S1 at end of FSM iteration, S3 sync before final `irq_o.write(true)` . Because the zeroer can saturate AXI for milliseconds, the typical run sees `quantum_exhausted` syncs dominate over `s4` syncs.

16. Mailbox Unit

Overview

Up to 32 bidirectional pairs (64 instances). Each pair: 2 KB outbound + 2 KB inbound. AXI4-Lite, 64-bit data. Independent IRQ per direction; threshold-based + overflow/underflow.

TLM-2.0 interface

```
template <int NUM_MB = 32>
SC_MODULE(mailbox_unit) {
    tlm_utils::simple_target_socket<mailbox_unit, 64> socket;
    sc_vector<sc_out<bool>> inbound_irq_o{NUM_MB};
    sc_vector<sc_out<bool>> outbound_irq_o{NUM_MB};
};
```

Per-mailbox register interface (offset within 2 KB)

Offset	Reg	Access	Notes
0x00	<code>WRITE_DATA</code>	WO	push to FIFO
0x08	<code>READ_DATA</code>	RO	pop from FIFO
0x10	<code>STATUS</code>	RO	empty/full/threshold
0x18	<code>ERROR_FLAGS</code>	RO	overflow/underflow
0x20	<code>WIRQT</code>	RW	write threshold
0x28	<code>RIRQT</code>	RW	read threshold
0x30	<code>IRQS</code>	RW1C	interrupt status
0x38	<code>IRQEN</code>	RW	enable
0x40	<code>IRQP</code>	RO	pending
0x48	<code>CTRL</code>	WO	flush

Address layout: `Outbound[N]` at `N*0x1000` ; `Inbound[N]` at `N*0x1000 + 0x800` .

Modeling notes

- FIFO implemented as `std::deque<uint64_t>` with depth `MAILBOX_DEPTH=8` (configurable).
- IRQs re-evaluated on every register access via `update_irqs()` .

17. System Timer OCTS

Overview

64-bit timer with PRIMARY/SECONDARY modes; credit-based synchronization. APB4/AXI4-Lite control (32 B). Generates `sync_load` and `credit` pulses (PRIMARY) or consumes them (SECONDARY).

TLM-2.0 interface

```
SC_MODULE(system_timer_oct) {
    tlm_utils::simple_target_socket<system_timer_oct> reg_socket;
    sc_in<bool>    is_primary_i;
    sc_in<bool>    timer_sync_load_i;
    sc_in<bool>    timer_cnt_credit_i;
    sc_in<sc_uint<8>> timer_cnt_step_i;
    sc_out<bool>    timer_sync_load_o;
    sc_out<bool>    timer_cnt_credit_o;
    sc_out<sc_uint<64>> timer_count_o;
};
```

Register interface (32 B)

Offset	Reg	Reset
0x00	<code>TIMER_START</code>	0
0x04	<code>CTRL</code> (CREDIT_VAL, PULSE_WIDTH)	0x0001020A
0x08	<code>STATUS</code> (RUNNING, MODE)	RO
0x0C	<code>TIMER_PRESET_LO</code>	0
0x10	<code>TIMER_PRESET_HI</code>	0
0x14	<code>TIMER_COUNT_LO</code>	0
0x18	<code>TIMER_COUNT_HI</code>	0
0x1C	<code>CREDIT_EXPIRED</code>	0

Modeling notes

- PRIMARY: `SC_THREAD` increments counter every `clk_ref_period_ns`; emits `sync_load_o` on START write, emits `credit_o` every `CREDIT_VAL` cycles.
- SECONDARY: counter advances by `step` while `cur_credits < CREDIT_VAL`; resets `cur_credits` on inbound pulses.
- 64-bit register accesses split into LO/HI as per spec.

18. eFuse

Overview

Layered model: interface controller + read/program FSMs + shadow registers (768 × 32-bit = 3 KB) + bank model + SHIM. Two access paths (SMC AXI-Lite + JTAG AXI-Lite) with lifecycle-based access control.

TLM-2.0 interface

```
SC_MODULE(efuse) {
    tlm_utils::simple_target_socket<efuse> smc_axi;    // BASE + 0x000_C000
    tlm_utils::simple_target_socket<efuse> jtag_axi;    // separate
    sc_in<bool> security_disable_i;
    sc_in<bool> secure_tm_i;
    sc_in<sc_uint<4>> lc_state_smc_i;
    sc_out<bool> fuse_sense_done_o;
    // 768 x 32-bit shadow output:
    sc_vector<sc_out<sc_uint<32>>> shadow_regs_o{768};
};
```

Register interface

Offset (BASE+0x000_B000)	Block
0x000–0xBFF	eFuse map (768 × 32-bit shadow)
0xC000–0xC0FF	eFuse Interface (CSR)
0xC100–0xC1FF	eFuse SHIM CSR
0xC200–0xC3FF	MMR (token-controlled)

Key registers: `EF_CMD`, `EF_ADDR`, `EF_WDATA`, `EF_RDATA`, `EF_STATUS`, `EF_LOCK`, `EF_BIRA[16-kbit]`, `RMA_SOP_TOKEN`, `RMA_CHIPILET_TOKEN`, `SECURITY_DISABLE_TOKEN`.

Modeling notes

- The internal storage is a `std::array<uint32_t, 768>`. **Set-once** semantics: writes are bit-level OR (programmed bits cannot be cleared).
- LC-state filter implemented in the JTAG path: when `lc_state ∈ {PROD, RMA_SOP}`, JTAG writes are blocked and reads return `0xBADCAB1E` except for chiplet-ID registers.
- `security_disable_i==1` bypasses sense FSM and exposes shadow regs immediately.

19. UART 16550

Overview

Industry-standard 16550 UART. AXI4-Lite, 64 B register space, configurable TX/RX FIFO depth (4–4096). 6-level prioritized interrupts; modem control; DMA ready signals.

TLM-2.0 interface

```
template <int TXD = 16, int RXD = 16>
SC_MODULE(uart_16550) {
    tlm_utils::simple_target_socket<uart_16550> reg_socket;
    sc_out<bool> irq_o;
    sc_out<bool> err_o;
    sc_out<bool> txrdy_o;
    sc_out<bool> rxrdy_o;
    // Serial pads (model only, drives stdout / pipe in TB)
    sc_out<bool> tx_o;
    sc_in<bool> rx_i;
    // Modem signals omitted in summary
};
```

Register interface (DLAB-multiplexed)

Offset	DLAB=0	DLAB=1
0x00	RBR (R) / THR (W)	DLL
0x04	IER	DLM
0x08	IIR (R) / FCR (W)	—
0x0C	LCR	LCR (DLAB)
0x10	MCR	—
0x14	LSR	—
0x18	MSR	—
0x1C	SCR	—
0x20	ECR	—
0x24	ITR	—

Modeling notes

- TX FIFO drained inside `SC_THREAD` at the configured baud rate (mapped to wall-clock or `sc_time`).
- For test convenience the model exposes `attach_stdout()` and `attach_socket(int port)` helpers.

20. Log Engine

Overview

DMA-based logging engine. 16 entries with round-robin arbitration. AXI4-Lite control (128 B) + 64-bit AXI4-Lite log fetch + 32-bit AXI4-Lite log write to UART.

TLM-2.0 interface

```
SC_MODULE(log_engine) {
    tlm_utils::simple_target_socket<log_engine> ctrl_socket;
    tlm_utils::simple_initiator_socket<log_engine> fetch_socket; // 64-bit
    tlm_utils::simple_initiator_socket<log_engine> write_socket; // 32-bit
    sc_in<bool> uart_tx_ready_i;
    sc_out<bool> irq_o;
};
```

Register interface

Address	Reg
0x00	CTRL
0x04	LOG_REGION_SIZE
0x08/0x0C	LOG_REGION_ADDR (64-bit)
0x10	LOG_WRITE_ADDR
0x14	INTR_STATUS
0x18	INTR_ENABLE
0x1C	INTR_TEST
0x40-0x7C	LOG_CTRL[0..15]

Modeling notes

- Round-robin arbiter implemented with a simple `next_index` cursor.
- Each fetched 64-bit word is split into 8 bytes and written one-by-one to `LOG_WRITE_ADDR` whenever `uart_tx_ready_i==true`.
- LT: zero delay between fetches; gated only by UART ready signal.
- Temporal decoupling (Bucket A, see §0):** `delay += descriptor_ns` per record consumed from the firmware ring; S2 sync before `wait(uart_tx_ready_event)`; S3 sync before any threshold IRQ. The log engine is the most TD-sensitive of the Bucket-A IPs because it back-pressures on a Bucket-D UART — leaving any sync out makes the trace appear to "leak" log records past the simulated time of subsequent firmware actions.

21. I2C Controller

Overview

OpenTitan-based I2C/SMBus/PMBus controller. AXI4-Lite (4 KB). Modes: controller, target, controller+target, monitor. 100 kHz / 400 kHz / 1 MHz. Configurable FMT/RX/ACQ/TX FIFOs.

TLM-2.0 interface

```
SC_MODULE(i2c_controller) {
    tlm_utils::simple_target_socket<i2c_controller> reg_socket;
    sc_out<bool> irq_fmt_threshold_o;
    sc_out<bool> irq_rx_threshold_o;
    sc_out<bool> irq_acq_threshold_o;
    sc_out<bool> irq_nak_o;
    sc_out<bool> irq_other_o;
    // Pad model
    sc_inout<bool> scl_io;
    sc_inout<bool> sda_io;
};
```

Register interface (key)

Offset	Reg
0x00	INTR_STATE
0x04	INTR_ENABLE
0x08	INTR_TEST
0x10	CTRL (mode, enable)
0x14	STATUS
0x18	RDATA
0x1C	FDATA (FMT FIFO write)
0x20	FIFO_CTRL
0x24	FIFO_STATUS
0x28	OVRD
0x2C–0x3C	Timing params (T_R, T_F, THIGH, TLOW, etc.)
0x40	TIMEOUT_CTRL
0x44	TARGET_ID

Modeling notes

- Bus model: a separate `i2c_bus` SC_MODULE arbitrates SCL/SDA as a wired-AND.
- LT: bytes transferred at the rate implied by `THIGH+TLOW` translated to `sc_time`. Each transfer modeled atomically.
- **Temporal decoupling (Bucket B, see §0)**: quantum capped by the byte-time computed from `THIGH+TLOW`. S3 sync mandatory on every IRQ output (`irq_fmt_threshold`, `irq_rx_threshold`, `irq_acq_threshold`, `irq_nak`, `irq_other`). Standard / Fast / Fast-mode-Plus all give per-byte times comfortably above the 1 μ s global quantum, so I2C effectively syncs per byte by construction.

22. AVSBus Controller

Overview

AVSBus 1.3.1 controller. APB4/AXI4-Lite (256 B). 8-deep command + 8-deep response FIFO. CRC-3 engine. 32-bit frame format. Single-target.

TLM-2.0 interface

```
SC_MODULE(avsbus_controller) {
    tlm_utils::simple_target_socket<avsbus_controller> reg_socket;
    sc_in<bool>    clk_ref_i;
    sc_in<bool>    rst_clk_div_n_i;
    sc_out<bool>   avs_clock_o;
    sc_out<bool>   avs_mdata_o;
    sc_in<bool>    avs_sdata_i;
    sc_out<bool>   irq_o;
};
```

Register interface

Address	Reg
0x00	AVS_CMD (W)
0x04	AVS_READBACK (R)
0x08	AVS_DEBUG_READBACK (R)
0x0C	AVS_LATEST_SLAVE_SUBFRAME (R)
0x20	AVS_NORMAL_STATUS
0x24	AVS_SLAVE_STATUS
0x28	AVS_FIFOS_STATUS
0x30	AVS_INTERRUPT
0x34	AVS_INTERRUPT_MASK
0x38	AVS_INTERRUPT_CLEAR
0x50	AVS_CFG_0 (retries, resync)
0x54	AVS_CFG_1 (clock cfg)
0x58	AVS_CONFIG

Modeling notes

- Frame TX/RX is functional: a connected `avs_target_model` SC_MODULE consumes/produces frames over the GPIO-like TLM interface.
- CRC-3 implemented as a 32-bit lookup function; mismatches trigger retry up to `AVS_CFG_0.MAX_RETRIES`.

23. GPIO Peripheral

Overview

68 GPIO instances (64 bonded + 4 unbonded). Each instance has a 16-byte interface register + 32-byte control register. AXI4-Lite. Strap capture on reset deassertion. Programmable interrupts (level/edge).

TLM-2.0 interface

```
SC_MODULE(gpio) {
    tlm_utils::simple_target_socket<gpio> intf_socket;
    tlm_utils::simple_target_socket<gpio> ctrl_socket;
    sc_vector<sc_inout<bool>> pad{NUM_GPIO}; // bidirectional
    sc_vector<sc_in<bool>> lsio_select_i{NUM_GPIO};
    sc_vector<sc_in<bool>> lsio_data_i{NUM_GPIO};
    sc_vector<sc_in<bool>> lsio_en_n_i{NUM_GPIO};
    sc_out<bool> irq_o; // OR of all 68
    sc_vector<sc_out<bool>> captured_strap_o{NUM_GPIO};
};
```

Register interface (per pin)

Interface register (16 B) | Offset | Reg | |---|---| | 0x00 | **DATA_CTRL** (interface_enable, enable_rx_tx, core2pad, pad2core, lsio_*, interrupt_type, interrupt_enable, strap_valid, strap_value) | | 0x04 | **ACCESS_FILTER** (awprot/arprot req, write/read filter en) | | 0x08 | **INT_STATUS** | | 0x0C | **INT_CLEAR** |

Control register (32 B) | Offset | Reg | |---|---| | 0x00 | **CONTROL** (drive_strength, pull_up, pull_down, config_enable, strap_validity) | | 0x04 | **STATUS** |

Modeling notes

- Strap capture: on rising edge of **rst_n_i**, sample **pad[N]** into **captured_strap_o[N]** and **DATA_CTRL.strap_value/valid**.
- Pad model: simple **bool** resolution. Drive contention not flagged in LT.

24. Telemetry Receiver

Overview

ATB telemetry receiver. AXI4-Lite (128 B). Decodes Tenstorrent telemetry messages (probe ID + up to 4 counters per msg). Configurable circular buffer (default 8 entries).

TLM-2.0 interface

```
SC_MODULE(telemetry_receiver) {
    tlm_utils::simple_target_socket<telemetry_receiver> reg_socket;
    // ATB sink:
    sc_in<sc_uint<8>> atdata_i;
    sc_in<sc_uint<7>> atid_i;
    sc_in<bool> atvalid_i;
    sc_out<bool> atready_o;
    sc_out<bool> afvalid_o;
    sc_in<bool> afready_i;
    sc_out<bool> irq_o;
};
```


Register interface

Offset	Reg
0x00	CTRL
0x04	STATUS
0x08	INTR_STATUS
0x0C	INTR_ENABLE
0x10	INTR_TEST
0x14	TELEMETRY_PROBE_ID
0x18	TELEMETRY_COUNTER_VLDS
0x80-0xFC	TELEMETRY_COUNTER[0..31]

Modeling notes

- Assembly buffer: `std::array<uint8_t,8>` accumulates 8 ATB beats into a 64-bit packet.
- Message decoder: parses 9-bit blocks for probe ID and up to 4 counters.
- Circular buffer implemented as `std::deque<telemetry_msg_t>`.

25. PVT Wrapper

Overview

Process / Voltage / Temperature monitoring block. 4 KB at BASE + 0x000_7000. AXI4-Lite. Provides temperature interrupt source (PLIC #283 via aggregator).

TLM-2.0 interface

```
SC_MODULE(pvt_wrapper) {
    tlm_utils::simple_target_socket<pvt_wrapper> reg_socket;
    sc_out<bool> temp_irq_o;
    sc_out<sc_int<16>> temperature_c_o;    // signed °C×10
    sc_out<sc_uint<16>> voltage_mv_o;
};
```

Register interface (subset)

Offset	Reg
0x000	PVT_CTRL
0x004	PVT_STATUS
0x010	TEMP_VALUE
0x014	VOLT_VALUE
0x018	PROC_VALUE
0x020	TEMP_THRESHOLD_HI
0x024	TEMP_THRESHOLD_LO
0x028	INT_EN
0x02C	INT_STATUS

Modeling notes

- Stimulus-driven: TB calls `force_temperature(int)` to drive transient values.
- Interrupt asserted when temperature crosses thresholds.

26. I3C Controller (`i3ccore_wrap` , x6)

Overview

The OCAH SMC integrates **6 instances** of the `i3ccore_wrap` controller (parameter `smc_config_pkg::NUM_I3C = 6`). The wrap is a three-level hierarchy in RTL:

```
i3ccore_wrapper    // top-level multi-instance demux (AXI4-Lite address decode)
└ i3c_wrapper      // per-instance CSR + signal routing
  └ i3c.sv         // HCI queues, FSMs, PHY, DAT/DCT
```

The controller is **MIPI I3C Basic v1.0/v1.1.1**, **HCI v1.2** and **TCRI v1.0** (recovery) compliant, originally based on the **CHIPS Alliance i3c-core** with OCA-specific enhancements (notably reduced-latency IBI during broadcast). Each instance supports active-controller (master), secondary-controller and target (slave) modes, plus full **I²C backwards compatibility** (Standard / Fast / Fast-mode-Plus).

In the SMC the canonical role split is:

Instance	Role
<code>i3c[0]</code>	Board-level communication (BMC/PMIC/sensors that prefer I3C)
<code>i3c[1]</code> , <code>i3c[2]</code>	Inter-chiplet communication
<code>i3c[3]</code> – <code>i3c[5]</code>	Spare / expansion

Bus characteristics:

- **AXI4-Lite slave**, 32-bit accesses **only** (write strobe must be `4'hF`).
- Per-instance address space: **0x500 bytes** (11-bit aperture, 0x000–0x4FF), 32-bit aligned.
- Multi-instance stride: **0x500** between consecutive instances.
- SMC base for the I3C aperture: **BASE + 0x040_0000** (within the 4 MB reserved region).

- Each instance asserts one IRQ; the 6 IRQs are routed to PLIC peripheral sources [17:12] after CDC from clk_periph_i to clk_smc_i.
- Push-Pull I3C up to **12.5 MHz**; Open-Drain for I²C (100 kHz / 400 kHz / 1 MHz).

TLM-2.0 interface

Two-level SystemC modeling that mirrors the RTL hierarchy:

```
// Per-instance controller (matches RTL `i3c_wrapper` / `i3c`)
SC_MODULE(i3c_controller) {
    tlm_utils::simple_target_socket<i3c_controller> reg_socket; // AXI4-Lite, 0x500 bytes

    sc_in<bool> rst_n_i; // periph reset
    sc_out<bool> irq_o; // -> PLIC peripheral source

    // Recovery (TCRI) sideband to the reset_unit / firmware mgr
    sc_out<bool> recovery_payload_available_o;
    sc_out<bool> recovery_image_activated_o;
    sc_out<bool> peripheral_reset_o;
    sc_out<bool> escalate_reset_o;

    // I3C bus, modelled as a multi-droop wired-AND like the i2c bus model.
    // The SystemC i3c_bus arbitrates SCL/SDA/OD-vs-PP across one controller and N targets.
    sc_port<i3c_bus_if> bus_port;

    // Modeling parameters
    unsigned cmd_q_depth = 8;
    unsigned resp_q_depth = 8;
    unsigned tx_q_depth = 64; // DWORDS
    unsigned rx_q_depth = 64; // DWORDS
    unsigned ibi_q_depth = 8;
    bool target_support = true;
    bool standby_controller = false;
    bool recovery_support = true;
};

// Top-level wrap (matches RTL `i3ccore_wrapper`): AXI4-Lite address-decode demux
template<unsigned N = 6>
SC_MODULE(i3c_controller_wrap) {
    tlm_utils::simple_target_socket<i3c_controller_wrap> reg_socket; // BASE+0x040_0000, N*0x500
    sc_vector<i3c_controller> inst{"i3c", N};
    sc_vector<sc_in<bool>> rst_n_i{"rst_n_i", N};
    sc_vector<sc_out<bool>> irq_o{"irq_o", N}; // [N-1:0] -> PLIC
};
```

The peripheral-clock-domain → SMC-clock-domain CDC for the IRQs and AXI request stream that exists in RTL is collapsed in LT modeling; we expose plain `sc_signal<bool>` IRQs and use a single `b_transport`. If a `clk_periph_i` quantum needs to be modelled (e.g. for AXI register access latency studies), the wrap exposes an optional `peripheral_clock_period` parameter.

Internal architecture

Per-instance modelling:

I3CBase (0x00–0x6C)

Offset	Reg	Acc	Reset	Notes
0x00	HCI_VERSION	R	0x0000_0120	HCI v1.2
0x04	HC_CONTROL	RW	0x0000_0040	BUS_ENABLE[31], RESUME[30], ABORT[29], HALT_ON_CMD_SEQ_TIMEOUT[12], HOT_JOIN_CTRL[8], I2C_DEV_PRESENT[7], MODE_SELECTOR[6]=1 (PIO, RO in OCA), IBA_INCLUDE[0] (ignored — OCA always sends 0x7E)
0x08	CONTROLLER_DEVICE_ADDR	RW	0	Dynamic address + valid bit for active-controller mode
0x0C	HC_CAPABILITIES	R	cfg	Read-only feature flags
0x10	RESET_CONTROL	RW1S/SC	0	IBI_QUEUE_RST[5], RX_FIFO_RST[4], TX_FIFO_RST[3], RESP_QUEUE_RST[2], CMD_QUEUE_RST[1], SOFT_RST[0] (self-clearing)
0x14	PRESENT_STATE	R	hw	AC_CURRENT_OWN[2]
0x20	INTR_STATUS	RW1C	0	SCHED_CMD_MISSED_TICK[14], HC_ERR_CMD_SEQ_TIMEOUT[13], HC_WARN_CMD_SEQ_STALL[12], HC_SEQ_CANCEL[11], HC_INTERNAL_ERR[10]
0x24	INTR_STATUS_ENABLE	RW	0	Per-source enable mask
0x28	INTR_SIGNAL_ENABLE	RW	0	Routing to irq_o
0x2C	INTR_FORCE	W	0	TB-only, software interrupt injection
0x30	DAT_SECTION_OFFSET	R	hw	DAT base + size
0x34	DCT_SECTION_OFFSET	RW	hw	DCT base + size + ENTDAI index
0x3C	PIO_SECTION_OFFSET	R	0x80	

PIOControl (0x80–0xAC)

Offset	Reg	Acc	Notes
0x80	<code>COMMAND_PORT</code>	W	64-bit cmd descriptor — two sequential 32-bit writes , second write enqueues
0x84	<code>RESPONSE_PORT</code>	R	32-bit resp: <code>[31:28]=err</code> , <code>[27:16]=len</code> , <code>[3:0]=TID</code>
0x88	<code>TX_DATA_PORT</code> / <code>RX_DATA_PORT</code>	W / R	Multiplexed by access direction; little-endian DWORD
0x8C	<code>IBI_PORT</code>	R	First read after <code>IBI_STATUS_THLD_STAT</code> returns the IBI status descriptor; subsequent reads return the IBI payload
0x90	<code>QUEUE_THLD_CTRL</code>	RW	<code>IBI_STATUS_THLD[31:24]</code> , <code>IBI_DATA_SEGMENT_SIZE[23:16]</code> , <code>RESP_BUF_THLD[15:8]</code> , <code>CMD_EMPTY_BUF_THLD[7:0]</code>
0x94	<code>DATA_BUFFER_THLD_CTRL</code>	RW	<code>RX_START_THLD[26:24]</code> , <code>TX_START_THLD[18:16]</code> , <code>RX_BUF_THLD[10:8]</code> , <code>TX_BUF_THLD[2:0]</code> (encoded as $2^{(N+1)}$)
0x98	<code>QUEUE_SIZE</code>	R	FIFO depths from RTL params
0x9C	<code>ALT_QUEUE_SIZE</code>	R	Extended IBI queue / alt resp queue size
0xA0	<code>PIO_INTR_STATUS</code>	RW1C	<code>CMD_QUEUE_READY</code> , <code>RESP_READY</code> , <code>TX_THLD_STAT</code> , <code>RX_THLD_STAT</code> , <code>IBI_STATUS_THLD_STAT</code> , <code>TRANSFER_COMPLETE</code> , <code>TRANSFER_ABORT</code> , <code>TRANSFER_ERR</code>
0xA4	<code>PIO_INTR_STATUS_ENABLE</code>	RW	
0xA8	<code>PIO_INTR_SIGNAL_ENABLE</code>	RW	

I3C_EC (Extended Capabilities, 0x100+)

Offset	Block	Notes
0x100	Standby Controller header (<code>STBY_CR_EXTCAP_HEADER</code> , CAP_ID=0x12)	
0x104	<code>STBY_CR_CONTROL</code>	<code>STBY_CR_ENABLE_INIT[31:30]</code> , DAA enables (<code>ENTDAA</code> / <code>SETDASA</code> / <code>SETAASA</code>), <code>TARGET_XACT_ENABLE</code> , <code>RSTACT_DEFBYTE_02</code>
0x108	<code>STBY_CR_DEVICE_ADDR</code>	Dynamic + static address
0x180+	Target Transaction Interface (TTI)	RX/TX queue mirrors when in target mode
0x200+	Recovery (TCRI)	Recovery command + payload registers
0x280+	SoC management	OCA-defined

DAT (0x300–0x3FC) and DCT (0x400–0x4FC)

- **DAT** — 128 entries × 2 DWORDs. Per-entry: `dynamic_addr[6:0]`, `static_addr[6:0]`, `IBI_EN`, `IBI_PAYLOAD_EN`, `dct_index`, `Push-Pull supported`. Looked up by `dev_idx` from the command descriptor.
- **DCT** — 128 entries × 4 DWORDs. Per-entry: `BCR`, `DCR`, `MWL`, `MRL`, `IBI_PAYLOAD_SIZE`, PID (48-bit). Populated by software from `GETPID` / `GETBCR` / `GETDCR` CCC responses.

Command descriptor (64-bit, written via `COMMAND_PORT`)

Bits	Field
[6:0]	Command Attribute (private write/read, CCC, address assignment)
[11:7]	<code>dev_idx</code> into DAT
[15:12]	Command Type (private / CCC / HDR)
[31:16]	Transfer length in bytes
[39:32]	CCC defining byte (when applicable)
[63:40]	Reserved / command-specific

Response descriptor (32-bit, read via `RESPONSE_PORT`)

Bits	Field
[3:0]	TID
[11:8]	Error info
[27:16]	Bytes actually transferred
[31:28]	Error status: <code>0x0=OK</code> , <code>0x1=CRC</code> , <code>0x2=PARITY</code> , <code>0x3=FRAME</code> , <code>0x4=ADDR_NACK</code> , <code>0x7=ABORT</code> , <code>0x8=I2C_W_NACK</code> , <code>0x9=I2C_R_NACK</code> , <code>0xF=OVF/UNF</code>

Transaction flows (LT modelling)

Each flow is implemented as a `SC_THREAD` inside `flow_active` that consumes one command descriptor and runs to completion before fetching the next.

Common init (every controller transaction)

```
IDLE → WaitStartTrig (TX/RX threshold OK?)  
  → FetchDAT      (look up dev_idx)  
  → BroadcastAddr (send 0x7E + W in 0D)  
  → <Write | Read | CCC | AddrAssign | IBI>
```

Controller write

`TargetAddr+W → FetchTxData → I3CDataWrite (per-byte loop) → WriteResp → IDLE` . NACK on `TargetAddr` → `ERROR_NACK` .
TX-FIFO underflow → `ERROR_UNDERFLOW` .

Controller read

`TargetAddr+R → I3CDataRead → PushRxData (loop) → WriteResp → IDLE` . RX-FIFO overflow → `ERROR_OVERFLOW` .

IBI handling

Two entry points:

1. **Bus Available Timer** (standard) — target initiates START on idle bus.
2. **Interrupt-during-broadcast** (OCA enhancement) — target asserts its own address during the controller's `BroadcastAddr` (wired-AND on SDA causes a mismatch). Controller saves `flow_active` state, services the IBI, then resumes the interrupted transaction. *Not supported by the Cadence I3C controller — must be modelled.*

`ReadTargetAddr → (I3CDataRead → PushRxData)* → WriteResp (IBI status) → resume | IDLE` .

CCC processing

CCC_SendCCCCode → [DefByte] → broadcast | direct(write|read) | address-assignment(SETDASA/SETNEWDA) → WriteResp → IDLE .

For broadcast CCCs (e.g. `ENEC`, `DISEC`) all targets process in parallel via their `ccc.sv` model; in SystemC we walk the `i3c_bus`'s registered targets.

IRQ wiring

Source (per instance)	Target
<code>i3c[N].irq_o</code> (any of <code>INTR_STATUS</code> v enabled <code>PIO_INTR_STATUS</code> bits)	<code>pllc</code> peripheral source <code>12 + N</code> (i.e. <code>[17:12]</code>)
<code>recovery_payload_available_o</code> , <code>recovery_image_activated_o</code>	Firmware mgr (informational; modelled as <code>sc_signal<bool></code>)
<code>peripheral_reset_o</code> , <code>escalate_reset_o</code>	<code>reset_unit</code> (back-channel — drives a soft-reset request)

The CDC stages between `clk_periph_i` and `clk_smc_i` present in RTL (`smc_peripherals_cdc.sv`) are collapsed; for delay studies the wrap can take an optional `cdc_latency = 2 * clk_periph_period`.

Integration in `smc_top`

```
// In smc_top:
i3c_controller_wrap<6>      i3c_wrap{"i3c_wrap"};    // AXI4-Lite at BASE+0x040_0000
i3c_bus                     i3c_bus_inst[6];        // one bus per instance

// Constructor:
fabric.bind(i3c_wrap.reg_socket, AddrRange{0x040'0000, 0x040'1DFF});
for (unsigned n = 0; n < 6; ++n) {
    i3c_wrap.inst[n].bus_port.bind(i3c_bus_inst[n]);
    i3c_wrap.rst_n_i[n].bind(rstu.periph_rst_n[n]);
    ic.peripheral_irq_in[12 + n].bind(i3c_wrap.irq_o[n]); // PLIC sources [17:12]
}
```

The `i3c_bus` SystemC module is a wired-AND model of SCL/SDA + an `OD/PP` arbitration flag; it is the I3C analogue of the `i2c_bus` model used by §21.

Modeling notes

- **Where fidelity matters most:** HCI queue semantics, IBI handling (especially the OCA-enhanced in-broadcast IBI), CCC engine, DAT/DCT lookups, error/response descriptor encoding, recovery sideband. These directly drive firmware code paths.
- **Where it does not:** bit-level SCL/SDA timing and CDC. We model byte-time only; the LT model never schedules events at sub-μs granularity unless explicitly requested.
- **Bus model:** mirror the `i2c_bus` pattern. Targets register themselves with `i3c_bus::register_target(addr, callback)`; controller's PHY model calls into the bus, which arbitrates.
- **Test devices:** ship a small set of canned target models (`i3c_target_pmic`, `i3c_target_sensor`, `i3c_target_eeprom`, `i3c_target_recovery_peer`) for use by the test plan in `03_SMC_Test_Plan.md`.
- **Unit tests:** GoogleTest-driven sanity checks adapted from the four firmware tests in `fw/smc/tests/i3c_*` (loop-back, raw master, raw slave, read/write sanity) so we can replay the same firmware flows the silicon DV team uses.
- **Optional Cadence shim parity:** if the SoC chooses to swap in a Cadence I3C macro (`hw/smc/dv_shims/i3c/cdni3c_*`) instead of the open-source core, the only externally visible delta is the absence of the in-broadcast IBI mechanism. Keep this as a compile-time `#define I3C_VARIANT_CADENCE` switch in `i3c_controller`.
- **What is *not* modelled (out of scope for LT):** HDR-DDR/HDR-TS modes (controller advertises but does not use), DMA mode (RTL is PIO-only in OCA), per-bit I3C glitch filtering, scheduled-command engine.
- **Temporal decoupling (Bucket B, see §0):** the `flow_active` SC_THREAD owns one `tlm_quantumkeeper qk_` per instance. Local time advances by `byte_time_pp` (PP) or `byte_time_od` (OD/I²C) per byte transferred on the wire. S1 at end of each byte loop, S2

before `wait(bus_event)` (controller idle / IBI detected on broadcast), S3 mandatory before `irq_o.write(true)` for `RESP_READY`, `IBI_STATUS_THLD`, and the recovery sideband signals — without S3 the firmware-measured IBI latency is off by one byte-time and the OCA in-broadcast IBI optimisation cannot be properly verified. Because `byte_time_od` ($\approx 25 \mu\text{s}$ at 400 kHz) exceeds the global quantum, OD/I²C transfers naturally sync per byte; PP transfers (`byte_time_pp  $\approx$  80 ns`) batch ~ 12 bytes per global-quantum window.

Top-level Integration: `smc_top`

```
SC_MODULE(smc_top) {
    // External AXI sockets
    tlm_utils::simple_target_socket<smc_top>    sys_axi_in;
    tlm_utils::simple_target_socket<smc_top>    sep_axi_in;
    tlm_utils::simple_target_socket<smc_top>    jtag_axi_in;
    tlm_utils::simple_initiator_socket<smc_top> output_axi;

    // Reset/clock parameters
    sc_in<bool>  rst_cold_n_i;
    sc_in<bool>  powergood_i;

    // Interrupt aggregation
    sc_out<bool> sync_irq_o;
    sc_vector<sc_out<bool>> ext_mailbox_irq_o{32};

    // GPIO pads
    sc_vector<sc_inout<bool>> pad{68};

    // ATB telemetry inputs
    sc_in<sc_uint<8>>  telemetry_atdata_i[3];
    sc_in<bool>        telemetry_atvalid_i[3];
    sc_out<bool>        telemetry_atready_o[3];

    // Sub-modules (P0/P1)
    smc_fabric        fabric;
    axi_filter         fin[16], fout[16];
    smc_cpu_cluster    cpu;
    plic               ic;
    clint              clintu;
    wdt                wdtu[4];
    bus_error_unit     beu[4];
    boot_rom            rom;
    scratchpad_sram     spm;
    reset_unit         rstu;
    pll_wrapper         pllu;
    misc_wrapper        miscu;
    mailbox_unit<32>     mb;
    system_timer_octs   octs;
    efuse               ef;
    uart_16550<>         uart[4];
    log_engine          log[4];
    i2c_controller      i2c[3];
    avsbus_controller   avs;
    gpio                gpiou;
    telemetry_receiver   trx[3];
    dma_engine           dma;
    memory_zeroer        zer;
    pvt_wrapper          pvt;
    debug_module         dm;
    i3c_controller_wrap<6> i3c_wrap;    // 6 I3C controllers (board / inter-chiplet / spare)

    // Constructor wires everything to the fabric and binds resets.
};
```

Address binding rules

- The fabric's downstream initiator sockets are bound to the appropriate IP target sockets in the constructor.
- Each filter is inserted between the fabric and a target by chaining `socket → filter.upstream → filter.downstream → target`.
- Resets are distributed via boolean signals issued by `reset_unit`.

Build & packaging

- C++17, `-std=c++17`, SystemC ≥ 2.3.4, TLM-2.0 standard headers.
- Build with CMake: `add_library(smc STATIC ...)`; tests use `add_executable(smc_tests ...)` linked to GoogleTest.
- Coding conventions: one IP per `<module>.h` + `<module>.cpp`. Register files generated to `<module>_reg.h`.

This document is the implementation contract for the SystemC/TLM-2.0 SMC IP library. The next document, `03_SMC_Test_Plan.md`, defines the full verification plan that exercises these modules individually and as an integrated `smc_top`.

Appendix A — ISS Integration Cookbook (Spike → SystemC TLM-2.0)

This appendix gives the implementation-level recipe for wrapping the **Spike** RISC-V ISS (`riscv-software-src/riscv-isa-sim`) and the RISC-V building blocks that ship with it (PLIC, CLINT, Debug Module, HTIF/FESVR, boot ROM, MMU/TLB) so they speak TLM-2.0 to the rest of the SMC model. The same pattern applies to **DBT-RISE-RISCV / TGC** and **Dromajo** — only the backend file changes.

The single design rule throughout: **everything Spike does that touches the outside world goes through one abstract interface (`iss_hart`) and one bus bridge (`iss_bus_bridge`)**. The SMC's own TLM IPs (fabric, PLIC, CLINT, mailbox, etc.) never link against Spike directly.

A.0 Reconciliation with the upstream Spike SystemC/TLM2.0 wrapper guide

The Spike repository ships its own integration note — `riscv-isa-sim/docs/systemc-tlm2-wrapper.md` — describing how the upstream community proposes to expose Spike to a SystemC/TLM-2.0 virtual platform. That guide and this appendix were written independently; they agree on every important architectural choice but use different names and have non-overlapping coverage in places. The table below is the authoritative cross-reference. **Where the two documents disagree, this appendix wins for the SMC project**, but downstream contributions back to Spike must use the upstream-proposed names so that a single Spike branch can serve every downstream user.

Topic	Upstream Spike doc (docs/systemc-tlm2-wrapper.md)	This appendix (SMC LLD)	Outcome / where reconciled
Top-level SC_MODULE name	<code>spike_tlm</code>	<code>smc_cpu_cluster</code> (which contains one Spike instance via the <code>spike_hart</code> backend)	The SMC adds a portability layer (<code>iss_hart</code> , §A.2) so Spike, TGC and Dromajo are interchangeable. The Spike-only <code>spike_tlm</code> module is a thin shim equivalent to instantiating <code>smc_cpu_cluster</code> with the Spike backend selected — see §A.11 build-system options.
Public Spike embedding API	<code>sim_t::start_embedding()</code> , <code>step_embedding(n)</code> , <code>done_embedding()</code> , <code>exit_code_embedding()</code>	Inferred — the backend reaches in via <code>processor_t::step()</code> directly	Adopted. §A.1 and §A.3 are updated to use the upstream embedding methods and to call out the small upstream patch they require.
MMIO bridging mechanism	One <code>tlm_bridge_device_t : public abstract_device_t</code> per memory window, registered with <code>bus_t</code>	<code>smc_simif::mmio_load/store</code> overrides forwarding to a single <code>iss_bus_bridge</code>	Both modes supported. §A.4 documents the trade-off and recommends the <code>tlm_bridge_device_t</code> flavour when the SMC needs Spike's <code>bus_t</code> to participate in DTB generation, and the <code>simif_t</code> override when it does not.
RAM ownership	Three options enumerated: (A) Spike-owned <code>mem_t</code> , (B) SystemC-owned via TLM bridge, (C) shared host-backed <code>abstract_mem_t::contents()</code>	DMI fast-path for ROM/SRAM (closest to A + DMI cache)	Reconciled in §A.5.1 , which states the SMC's per-region default: ROM = A+DMI, Scratchpad SRAM = A+DMI, Mailbox payload region = C, external NoC ranges = B.
Default quantum	<code>INTERLEAVE = 5000</code> instructions $\approx 5 \mu\text{s}$ at 1 GHz (Spike's CLI default; recommended unchanged for the upstream wrapper)	<code>quantum_insts_ = 1000</code> $\approx 1 \mu\text{s}$ at 1 GHz	Reconciled in §A.10 and §0.6. The SMC overrides to 1 μs because peripheral IRQ-latency budgets are tighter than Spike's stand-alone use case; the upstream 5 μs default remains valid for OS-boot regressions where IRQ latency is dominated by Linux scheduling.
WFI handling	<code>processor_t::is_waiting_for_interrupt()</code> + <code>clear_waiting_for_interrupt()</code> ; suggested policy: skip the quantum when all harts are in WFI	Not previously documented	Adopted in §A.10. The hart <code>SC_THREAD</code> now consults <code>is_waiting_for_interrupt()</code> and waits on <code>quantum_event_[id]</code> instead of burning instructions.
External-IRQ injection	<code>sim->get_intctrl()->set_interrupt_level(id, level)</code> (uses Spike's built-in PLIC) — recommended for the first milestone	Replaced by SMC TLM PLIC + <code>irq_aggregator()</code> (§A.6) for production; Spike's built-in PLIC kept available only as scaffold (§A.7.3)	Equivalent for the first milestone. §A.7.3 explicitly aliases the upstream first-milestone IRQ path.
Build integration	Optional <code>--with-systemc=<prefix></code> configure argument; <code>MCPPBS_SUBPROJECTS</code> adds a <code>systemc/</code> sub-project producing <code>libspike-systemc</code>	CMake <code>ExternalProject_Add</code> builds Spike's autotools out-of-tree and exposes <code>spike::riscv</code> etc.	Both supported. §A.11 documents both, with CMake as the SMC default and <code>--with-systemc</code> recommended for upstream contribution.

Topic	Upstream Spike doc (<code>docs/systemc-tlm2-wrapper.md</code>)	This appendix (SMC LLD)	Outcome / where reconciled
ELF loading / <code>tohost</code> / <code>fromhost</code>	FESVR; keep <code>htif_t::start()</code> but never call <code>htif_t::run()</code> from a SystemC process	Same — explicit in §A.8	Agree.
Endianness / access width	Preserve byte order, return TLM error for unsupported widths, big-endian opt-in (<code>--enable-dual-endian</code>) needs explicit byte-lane validation	Implicit	Adopted in §A.4. Explicit guidance added.
Validation plan	7 staged tests (ISA baseline → bare-metal+RAM-only → MMIO → PLIC IRQ → WFI → SMC firmware → SystemC-owned RAM)	Lives in <code>03_SMC_Test_Plan.md</code>	Cross-linked in §A.14.
Risks / pinned revision	"Spike's principal public API is the RISC-V ISA; internal C++ APIs may change incompatibly. Keep the SMC wrapper close to one pinned Spike revision."	Not previously documented	Adopted as §A.15. The SMC repo pins Spike via a git submodule + commit SHA.
First-implementation milestone	6-step list (add embedding methods → external <code>spike_tlm</code> → Spike-owned RAM → one bridge window → Spike PLIC → bare-metal ELF)	Not previously consolidated	Adopted as §A.14, with each step mapped to the corresponding sub-section of this appendix.

The remainder of Appendix A reflects this reconciliation. Sub-section headings have not changed (so existing internal references still resolve), but the bodies of §A.1, §A.3, §A.4, §A.7.3, §A.10 and §A.11 have been updated, and §A.5.1, §A.14 and §A.15 have been added.

A.1 Spike API surface that matters

When Spike is built as a library (`libriscv.a`) the headers used by the wrapper live in `riscv-isa-sim/install/include/riscv/`. The relevant symbols:

Header	Symbol	What we use it for
<code>processor.h</code>	<code>class processor_t</code>	One per hart; <code>step(n)</code> , <code>reset()</code> , <code>set_pc(pc)</code> , <code>set_mip(mask)</code> , <code>get_state()</code> , <code>get_csr(n)</code> , <code>set_csr(n,v)</code> , <code>is_waiting_for_interrupt()</code> , <code>clear_waiting_for_interrupt()</code> , <code>trigger_nmi()</code>
<code>sim.h</code>	<code>class sim_t</code>	Top-level container — used only when scaffold mode (§A.7.3) keeps Spike's built-in PLIC/CLINT/Debug Module. Production mode bypasses <code>sim_t</code> and drives <code>processor_t</code> directly through <code>spike_pool</code> (§A.3). When <code>sim_t</code> is used, the new <code>start_embedding()/step_embedding(n)/done_embedding()/exit_code_embedding()</code> methods (proposed in <code>riscv-isa-sim/docs/systemc-tlm2-wrapper.md</code>) are the bounded-step substitutes for <code>sim_t::run()</code> / <code>htif_t::run()</code>
<code>simif.h</code>	<code>class simif_t</code>	Abstract host that Spike asks for memory & MMIO; we implement this as <code>smc_simif</code>
<code>mmu.h</code>	<code>class mmu_t</code> , <code>MMIO_LOAD/STORE</code> macros	Spike's per-hart MMU forwards uncached/MMIO accesses to <code>simif_t</code>
<code>devices.h</code>	<code>class</code> <code>abstract_device_t</code> , <code>class</code> <code>abstract_mem_t</code> , <code>class mem_t</code> , <code>class</code> <code>bus_t</code> , <code>class plic_t</code> , <code>class clint_t</code> , <code>class</code> <code>debug_module_t</code> , <code>class</code> <code>external_sim_device_t</code>	Built-in PLIC/CLINT/DM are <i>replaced</i> (§A.7); <code>abstract_device_t</code> is the base class for the upstream-style <code>tlm_bridge_device_t</code> (§A.4); <code>mem_t</code> and a host-backed <code>abstract_mem_t</code> are used for the RAM-ownership choices in §A.5.1
<code>mmio_plugin.h</code>	<code>mmio_device_t</code> , <code>mmio_plugin_register</code>	Mechanism for hooking an external device to a memory range
<code>fesvr/elfloader.h</code>	<code>load_elf()</code>	ELF parser for firmware images
<code>fesvr/htif.h</code>	<code>htif_t</code>	Optional <code>tohost</code> / <code>fromhost</code> console; we keep it for early bring-up. Never call <code>htif_t::run()</code> from a SystemC process — it is blocking. Use <code>htif_t::start()</code> once and poll <code>htif_t::done()</code> between quanta.
<code>disasm/disasm.h</code>	<code>disassembler_t</code>	Optional commit-log disassembly for trace

Spike's build flags worth knowing:

```

./configure --enable-commitlog      # turns on per-instruction trace
            --enable-histogram      # PC histograms (debug)
            --enable-dual-endian     # only if SMC firmware ever runs BE
            --with-isa=rv64gc        # match SMC Rocket capabilities
            --with-priv=msu          # add 'h' if Xvisor required
            --with-systemc=$SYSTEMC_HOME # build libspike-systemc with the upstream wrapper (optional, see §A.11)

```

Required upstream patch — `sim_t` embedding methods

The upstream Spike wrapper guide identifies one piece of source-level surgery the SMC inherits when it ever uses `sim_t` (i.e. scaffold mode, §A.7.3, or any future co-sim configuration). Spike's normal entry-point — `int sim_t::run()` — delegates to the blocking `htif_t::run()` loop, which calls back into `sim_t::idle()` and never returns control to a SystemC thread until simulation completes. The fix is a small, additive set of public methods on `sim_t`:

```
// Patch to riscv/sim.h / riscv/sim.cc
class sim_t : public htif_t, public simif_t {
public:
    void start_embedding();           // = set_expected_xlen + htif_t::start
    void step_embedding(size_t n);    // bounded step; tick remote_bitbang once
    bool done_embedding() const;      // wraps htif_t::done()
    int  exit_code_embedding() const; // wraps htif_t::exit_code()
};
```

The implementation is a five-line wrap of existing private behaviour (full body in the upstream guide, §"Minimal Spike API Additions"). The SMC pins the Spike submodule to a revision that carries this patch (§A.15); until the patch lands upstream the SMC repo carries it as a small `0001-add-embedding-api.patch` applied by the `external/spike-cmake/` `ExternalProject` step.

A.2 The `iss_hart` abstract interface (one source of truth)

All ISS backends must implement this interface; the SystemC cluster knows nothing else.

```

// iss_hart.h (in libsmc/include/cpu/)
namespace smc {

struct iss_irq_state {
    bool meip = false, seip = false, mtip = false, msip = false;
    bool nmi = false, debug = false;
};

class iss_hart {
public:
    virtual ~iss_hart() = default;

    virtual void reset() = 0;
    virtual void set_pc(uint64_t pc) = 0;
    virtual uint64_t get_pc() const = 0;

    // Run up to `n` instructions; returns instructions retired.
    virtual uint32_t step(uint32_t n) = 0;

    // Pin-level IRQ injection; called between step()s by the SystemC thread.
    virtual void set_irqs(const iss_irq_state& s) = 0;

    // CSR back-door for the test-bench.
    virtual uint64_t read_csr(uint16_t csr) const = 0;
    virtual void write_csr(uint16_t csr, uint64_t v) = 0;

    // Bus-bridge hook: backend calls these for every load/store/fetch.
    // (Backend is responsible for routing through the bridge it was given.)
    using bus_bridge_t = std::function<bool(uint64_t addr, size_t len,
                                           uint8_t* data, bool is_write,
                                           uint8_t prot, uint8_t axi_id)>;
    virtual void attach_bridge(bus_bridge_t mem,
                              bus_bridge_t mmio,
                              bus_bridge_t ifetch) = 0;

    // WFI hooks ($A.10). Backends that don't model WFI explicitly may return
    // `false` from is_waiting_for_interrupt() to fall back to the unconditional
    // step policy.
    virtual bool is_waiting_for_interrupt() const { return false; }
    virtual void clear_wfi() {}

    // Optional commit-log entry for co-simulation.
    struct commit_entry { uint64_t pc, instr; uint8_t priv; std::vector<std::pair<int, uint64_t>> regs; };
    virtual std::optional<commit_entry> last_commit() const { return std::nullopt; }
};

std::unique_ptr<iss_hart> make_spike_hart(unsigned id, std::string isa, std::string priv);
std::unique_ptr<iss_hart> make_tgc_hart(unsigned id, std::string isa, std::string priv);
std::unique_ptr<iss_hart> make_dromajo_hart(unsigned id, std::string isa, std::string priv);

} // namespace smc

```

Switching ISS = changing which `make_*_hart()` factory is called in `smc_cpu_cluster`'s constructor. Nothing else moves.

A.3 Implementing the Spike backend (`iss_backend_spike.cpp`)

The wrapper has three classes:

1. `smc_simif` — implements Spike's `simif_t`. All Spike memory traffic enters here.
2. `spike_hart` — implements `iss_hart` by owning a `processor_t`.
3. `spike_pool` — a process-wide singleton that owns a `cfg_t`, the `simif`, and the vector of harts (Spike requires shared state across harts for `mtime`/AMOs).

```

// iss_backend_spike.cpp
#include "cpu/iss_hart.h"
#include <riscv/processor.h>
#include <riscv/simif.h>
#include <riscv/cfg.h>

namespace smc {

class smc_simif : public simif_t {
public:
    using bridge_t = iss_hart::bus_bridge_t;
    smc_simif(bridge_t mem, bridge_t mmio, bridge_t ifetch)
        : mem_(std::move(mem)), mmio_(std::move(mmio)), ifetch_(std::move(ifetch)) {}

    // simif_t interface -----
    char* addr_to_mem(reg_t addr) override { return nullptr; } // no host pointer
    bool reservable (reg_t addr) override { return true; } // allow LR/SC anywhere
    bool mmio_load (reg_t a, size_t n, uint8_t* b) override
        { return mmio_(a, n, b, false, cur_prot_, cur_id_); }
    bool mmio_store(reg_t a, size_t n, const uint8_t* b) override
        { return mmio_(a, n, const_cast<uint8_t*>(b), true,
            cur_prot_, cur_id_); }
    void proc_reset (unsigned) override {}
    const cfg_t& get_cfg() const override { return cfg_; }
    const std::map<size_t, processor_t*>& get_harts() const override { return hart_map_; }
    const char* get_symbol(uint64_t addr) override { return nullptr; }

    // Hook used by spike_hart::step() before/after to set context bits.
    void set_xact_ctx(uint8_t prot, uint8_t id) { cur_prot_ = prot; cur_id_ = id; }

    // Used by FESVR ELF loader.
    bool host_write(reg_t a, size_t n, const uint8_t* b)
        { return mem_(a, n, const_cast<uint8_t*>(b), true, 0xF, 0); }

private:
    bridge_t mem_, mmio_, ifetch_;
    cfg_t cfg_; // populated by spike_pool
    std::map<size_t, processor_t*> hart_map_;
    uint8_t cur_prot_ = 0, cur_id_ = 0;
    friend class spike_pool;
};

class spike_hart : public iss_hart {
public:
    spike_hart(unsigned id, std::string isa, std::string priv, smc_simif* sim)
        : id_(id), proc_(&sim->get_cfg(), sim, id, /*halt_on_reset*/ true,
            /*log_file*/ nullptr, /*sout*/ std::cerr) {
        proc_.set_isa(isa);
        proc_.set_privilege_level(priv);
    }

    void reset() override { proc_.reset(); }
    void set_pc(uint64_t pc) override { proc_.get_state()->pc = pc; }
    uint64_t get_pc() const override { return proc_.get_state()->pc; }

    uint32_t step(uint32_t n) override {
        sim_->set_xact_ctx(compute_prot(), static_cast<uint8_t>(id_));
        proc_.step(n);
        return n;
    }

    // Exposed so the cluster's hart_thread() can park on quantum_event_[id]
    // when all harts are in WFI rather than burn instructions in a tight loop.
    // Mirrors the upstream wrapper guide's recommended WFI policy (§A.10).
    bool is_waiting_for_interrupt() const { return proc_.is_waiting_for_interrupt(); }
};

```



```

void clear_wfi() { proc_.clear_waiting_for_interrupt(); }

void set_irqs(const iss_irq_state& s) override {
    reg_t mip = 0;
    if (s.meip) mip |= MIP_MEIP;
    if (s.seip) mip |= MIP_SEIP;
    if (s.mtip) mip |= MIP_MTIP;
    if (s.msip) mip |= MIP_MSIP;
    proc_.get_state()->mip->backdoor_write_with_mask(MIP_MEIP|MIP_SEIP|MIP_MTIP|MIP_MSIP, mip);
    if (s.nmi) proc_.trigger_nmi();
    if (s.debug) proc_.halt_request = true; else proc_.halt_request = false;
}

uint64_t read_csr(uint16_t csr) const override { return proc_.get_csr(csr); }
void write_csr(uint16_t csr, uint64_t v) override { proc_.set_csr(csr, v); }

void attach_bridge(bus_bridge_t mem, bus_bridge_t mmio, bus_bridge_t ifetch) override {
    sim_ = nullptr; // wired at construction by spike_pool
}

private:
uint8_t compute_prot() const {
    auto* st = proc_.get_state();
    bool priv_m = (st->prv == PRV_M);
    bool priv_s = (st->prv == PRV_S);
    return (priv_m ? 0x0 : (priv_s ? 0x2 : 0x6)); // {data, !secure, !priv}
}
unsigned id_;
smc_simif* sim_;
processor_t proc_;
};

} // namespace smc

```

A few subtleties worth calling out:

- `simif_t::addr_to_mem` returning `nullptr` forces every load/store through `mmio_load` / `mmio_store`. That's exactly what we want — even SRAM accesses traverse the bridge so the SMC fabric and the inbound/outbound filters see them. (The DMI fast-path in §A.5 restores the speed.)
- `mip->backdoor_write_with_mask` is a Spike helper that bypasses the normal CSR write side-effects so we can mirror the live PLIC/CLINT outputs without the hart having to execute a CSR write itself.
- `proc_.trigger_nmi()` exists in current Spike; if your version is older, set `mip` bit `MIP_NMIP` directly via the same back-door API.

A.4 Bus bridge: Spike memory ops → TLM `b_transport`

The bridge is a pure C++ class owned by `smc_cpu_cluster`. It does **not** inherit from `sc_module`; it is just a function object that captures the cluster's TLM sockets so it can be passed to the ISS backend through `iss_hart::attach_bridge`.

```

// iss_bus_bridge.h
class iss_bus_bridge {
public:
    iss_bus_bridge(tlm_utils::simple_initiator_socket<smc_cpu_cluster>& mmio,
                  tlm_utils::simple_initiator_socket<smc_cpu_cluster>& data,
                  tlm_utils::simple_initiator_socket<smc_cpu_cluster>& ifetch,
                  sc_core::sc_time& local_time)
        : mmio_(mmio), data_(data), ifetch_(ifetch), t_(local_time) {}

    bool xfer(uint64_t addr, size_t len, uint8_t* buf, bool is_write,
              uint8_t prot, uint8_t axi_id, channel ch);

private:
    enum class channel { MMIO, DATA, IFETCH };
    tlm_utils::simple_initiator_socket<smc_cpu_cluster>& mmio_, data_, ifetch_;
    sc_core::sc_time& t_;
};

bool iss_bus_bridge::xfer(uint64_t addr, size_t len, uint8_t* buf,
                          bool is_write, uint8_t prot, uint8_t axi_id,
                          channel ch) {
    tlm::tlm_generic_payload gp;
    smc_axi_extension* ext = new smc_axi_extension(); // pooled in real code
    ext->source_id = SMC_ID;
    ext->prot = prot;
    ext->non_secure = !(prot & 0x2);
    ext->axi_id = axi_id;
    gp.set_extension(ext);

    gp.set_command(is_write ? tlm::TLM_WRITE_COMMAND : tlm::TLM_READ_COMMAND);
    gp.set_address(addr);
    gp.set_data_ptr(buf);
    gp.set_data_length(len);
    gp.set_streaming_width(len);
    gp.set_byte_enable_ptr(nullptr);
    gp.set_dmi_allowed(false);
    gp.set_response_status(tlm::TLM_INCOMPLETE_RESPONSE);

    auto& sock = (ch == channel::MMIO) ? mmio_
        : (ch == channel::DATA) ? data_
        : ifetch_;
    sock->b_transport(gp, t_);

    return gp.get_response_status() == tlm::TLM_OK_RESPONSE;
}

```

Address routing — the cluster's constructor builds a small `flat_map<addr_range, channel>` from the SMC memory map:

Range	Channel
<code>0xC000_0000 .. 0xC000_FFFF</code> (Boot ROM, peripheral xbar entry, mailbox, OCTS, eFuse, UART, I2C, AVS, GPIO, Telemetry, PVT, Reset, MISC, PLL, WDT, BEU, DM)	MMIO
<code>0xC400_0000 .. 0xC43F_FFFF</code> (PLIC), <code>0xC800_0000</code> (CLINT)	MMIO
<code>0xC006_0000 .. 0xC00F_FFFF</code> (Scratchpad SRAM), <code>0xC004_0000 .. 0xC005_FFFF</code> (Boot ROM data window), high-perf NoC	DATA
Same as <code>DATA</code> but flagged with <code>prot[2]=1</code> (instruction)	IFETCH

The `IFETCH` channel exists so the SMC fabric can distinguish I-side from D-side transactions for security filtering and so DMI can be granted only on the I-side cache line, mirroring the Rocket I-cache.

A.4.1 Two equivalent ways of plugging the bridge into Spike

The upstream Spike wrapper guide proposes that the TLM bridge be implemented as a Spike `abstract_device_t` registered with `bus_t` for each MMIO window:

```
// upstream-style - riscv-isa-sim/docs/systemc-tlm2-wrapper.md
class tlm_bridge_device_t : public abstract_device_t {
public:
    tlm_bridge_device_t(reg_t base, reg_t size,
                       tlm::tlm_initiator_socket<>& socket,
                       sc_core::sc_time* local_delay);
    bool load(reg_t addr, size_t len, uint8_t* bytes) override;
    bool store(reg_t addr, size_t len, const uint8_t* bytes) override;
    reg_t size() override { return size_; }
};
```

For the SMC we instead override `simif_t::mmio_load` / `mmio_store` directly (§A.3) so that *all* non-RAM accesses funnel through one `iss_bus_bridge::xfer()` and a single TLM extension (`smc_axi_extension`) is constructed exactly once per access. The two approaches differ only in *who decodes the address*:

Aspect	<code>tlm_bridge_device_t</code> (upstream-style)	<code>simif_t::mmio_*</code> override (SMC default)
Address decode	<code>bus_t</code> inside Spike (one device per window, via <code>bus.add_device(base, dev)</code>)	<code>iss_bus_bridge</code> decodes from the SMC memory map
Visibility in DTB	Yes — Spike's <code>generate_dts()</code> walks <code>bus_t</code>	No — devices remain invisible to FDT generation
Number of integration points	One per memory window	One total
Required Spike patch	Fix <code>external_sim_device_t::size()</code> PGSIZE TODO, or just use <code>tlm_bridge_device_t</code> directly	None
Overhead per MMIO access	Two indirections (<code>bus_t</code> lookup + virtual call)	One virtual call

SMC default: `simif_t::mmio_*` override, because the SMC fabric and `axi_filter` already own address decode, and we never use Spike's DTB generator (the SMC has its own device tree pinned in firmware). **When to switch to `tlm_bridge_device_t`:** Linux-class boots that want Spike's DTB to enumerate SMC peripherals automatically, or any future co-sim configuration where Spike's `bus_t` is the single source of truth.

Both modes coexist in the same backend file behind a build-time flag (`SMC_SPIKE_BUS_MODE=simif | bus_t`); the compiled-in choice does not change anything visible in `iss_hart` or above.

A.4.2 Endianness, access width and TLM error semantics

The upstream guide is explicit about subword/byte-lane discipline; the SMC adopts the same rules:

- **Byte order is preserved end-to-end.** Spike supplies `uint8_t bytes[len]` already in target-memory order. The bridge does **not** cast through host integer types (except for logging) and copies byte arrays verbatim into `tlm_generic_payload::set_data_ptr()` . Big-endian targets (built with `--enable-dual-endian` and configured with `--big-endian`) work unchanged provided the SMC fabric and downstream peripherals also accept raw byte arrays — verify with explicit byte-lane regressions before signing off.
- **Subword `len` s:** TLM allows 1, 2, 4, 8-byte lengths. Anything else (e.g. an unaligned 3-byte access from a misbehaving load instruction) must produce `tlm::TLM_GENERIC_ERROR_RESPONSE` , which the bridge maps back to `false` from `mmio_load/store` . The MMU then reflects this as a load/store access fault, identically to a real bus error.
- **`set_streaming_width(len)`** is set equal to `data_length` ; we do not use TLM streaming bursts on the cluster initiator side because Spike issues one access per instruction.

- `set_byte_enable_ptr(nullptr)` on aligned accesses; subword *register* writes that need lane masking are emitted by the upstream peripheral, not by Spike.

These rules also let the bridge be reused unchanged when `tlm_bridge_device_t` is selected (§A.4.1).

A.5 DMI fast-path for ROM and Scratchpad SRAM

Going through `b_transport` for every Spike memory access is the slow path. The bridge therefore offers a DMI cache:

```
struct dmi_entry { uint8_t* ptr; uint64_t base, end; bool writable; };
std::vector<dmi_entry> dmi_cache_;

bool iss_bus_bridge::try_dmi(uint64_t addr, size_t len, uint8_t* buf, bool wr) {
    for (auto& e : dmi_cache_)
        if (addr >= e.base && (addr + len) <= e.end) {
            if (wr) { if (!e.writable) return false; std::memcpy(e.ptr+(addr-e.base), buf, len); }
            else { std::memcpy(buf, e.ptr+(addr-e.base), len); }
            return true;
        }
    return false;
}
```

On the first miss for an address the bridge issues a TLM `get_direct_mem_ptr()`; if the fabric returns `dmi_allowed=true` (boot ROM and scratchpad SRAM advertise DMI per §8 and §9 of this document), the descriptor is added to `dmi_cache_`. On reset and on any `invalidate_direct_mem_ptr` call from the fabric the cache is flushed.

This is the single most important performance optimisation — instruction fetch hits the DMI cache 99 %+ of the time once firmware is resident.

A.5.1 RAM ownership policy

The upstream Spike wrapper guide enumerates three valid RAM-ownership models. The SMC adopts a *per-region* combination of all three rather than a single global choice. The decision matrix:

Region	Default model	Why	Where modelled
Boot ROM (<code>0xC004_0000</code> ... <code>0xC005_FFFF</code>)	A — Spike-owned <code>mem_t</code> + DMI	Read-only, fits in one DMI page, accessed billions of times during early boot loops.	§8 + §A.5
Scratchpad SRAM (<code>0xC006_0000</code> ... <code>0xC00F_FFFF</code>)	A — Spike-owned <code>mem_t</code> + DMI (default) / B — SystemC-owned via <code>tlm_bridge_device_t</code> (strict-fabric mode)	Default A for boot speed; mode B is selectable for <code>03_SMC_Test_Plan.md</code> cases that exercise the inbound-filter <code>prot</code> / <code>source_id</code> path on every SRAM access (e.g. SEP write isolation tests).	§9, §A.4.1
Mailbox payload window	C — shared host-backed <code>abstract_mem_t</code>	Both Spike and the mailbox unit's TLM target need to see the same bytes; <code>contents()</code> returns a pointer into a <code>std::vector<uint8_t></code> owned by the mailbox SC_MODULE.	§10
External NoC ranges (<code>0x0000_0000</code> ... <code>0xBFFF_FFFF</code>)	B — SystemC-owned via the <code>data</code> socket	Spike must never speculate against external memory; every access has to traverse the inbound/outbound filters and possibly leave the chip.	§A.4
PLIC, CLINT, peripherals (MMIO)	n/a — handled by the bridge, no <code>abstract_mem_t</code> involved	All accesses go through <code>simif_t::mmio_*</code> (or <code>tlm_bridge_device_t</code>).	§A.4

Implementation notes:

- **Mode A — Spike-owned `mem_t`**. Used unchanged from `spike_main/spike.cc`:

```
std::vector<std::pair<reg_t, abstract_mem_t*>> mems;
mems.push_back({BOOT_ROM_BASE, new mem_t(BOOT_ROM_SIZE)});
mems.push_back({SCRATCHPAD_BASE, new mem_t(SCRATCHPAD_SIZE)});
```

These pointers are surfaced to the rest of the SMC via `boot_rom`'s and `scratchpad_sram`'s `get_direct_mem_ptr()` so the SystemC test bench can still poke at them through DMI. ELF loading writes through `host_write()` (§A.8), which uses the bridge's DATA channel, so address remap and ECC/init logic in scratchpad SRAM are *still* exercised at load time even though runtime accesses are direct.

- **Mode B — SystemC-owned via TLM bridge**. Either route the address range through the `iss_bus_bridge::xfer()` DATA channel (current SMC mechanism), or register a `tlm_bridge_device_t(base, size, data_socket, &local_delay)` with Spike's `bus_t` (upstream-style). Both reach the same SystemC peripheral. **Cost:** every load/store becomes a TLM transaction, so single-thread Spike performance drops by ~30–80× depending on access pattern. Use only for tests that *require* fabric visibility on every RAM access.
- **Mode C — shared host-backed RAM**. Implement `abstract_mem_t` with `contents()` returning a pointer into a `std::vector<uint8_t>` owned by a SystemC module:

```
class shared_host_mem : public abstract_mem_t {
public:
    explicit shared_host_mem(std::vector<uint8_t>& backing) : data_(backing) {}
    char* contents() override { return reinterpret_cast<char*>(data_.data()); }
    size_t size() const override { return data_.size(); }
    bool load(reg_t a, size_t n, uint8_t* b) override { /* memcpy */ }
    bool store(reg_t a, size_t n, const uint8_t* b) override { /* memcpy */ }
private:
    std::vector<uint8_t>& data_;
};
```

The SMC mailbox unit owns the `std::vector<uint8_t>`; both Spike (through `addr_to_mem`) and the mailbox's TLM target socket index into the same buffer. Because mailbox accesses are intrinsically a producer/consumer hand-off, the lack of TLM timing on Spike's side is acceptable — the mailbox SC_MODULE still raises an interrupt with the correct `delay`.

The choice between A, B and C for any *new* memory region added later is recorded in `01_SMC_Architecture.md` §2.2 (memory map), and the rationale follows the same three rules: speed-critical and read-mostly → A; needs strict fabric/filter visibility on every access → B; shared producer/consumer → C.

A.6 Interrupt aggregator: SMC IPs → Spike CSRs

The cluster's IRQ aggregator is a single `SC_METHOD` triggered by all of the IRQ inputs. It builds an `iss_irq_state` and pushes it into every hart between quanta:

```

SC_METHOD(irq_aggregator);
for (auto& p : meip_in)    sensitive << p.value_changed_event();
for (auto& p : seip_in)    sensitive << p.value_changed_event();
for (auto& p : mtip_in)    sensitive << p.value_changed_event();
for (auto& p : msip_in)    sensitive << p.value_changed_event();
for (auto& p : beu_nmi_in) sensitive << p.value_changed_event();
sensitive << debug_irq_in.value_changed_event();
dont_initialize();

void smc_cpu_cluster::irq_aggregator() {
    for (unsigned i = 0; i < harts_.size(); ++i) {
        iss_irq_state s {
            .meip = meip_in[i].read(),
            .seip = seip_in[i].read(),
            .mtip = mtip_in[i].read(),
            .msip = msip_in[i].read(),
            .nmi  = beu_nmi_in[i].read(),
            .debug = debug_irq_in.read(),
        };
        harts_[i]->set_irqs(s);
        quantum_event_[i].notify(SC_ZERO_TIME);    // wake hart to re-check immediately
    }
}

```

This decouples the SMC's TLM PLIC/CLINT/BEU from Spike entirely — they drive `sc_signal<bool>` exactly as they would for the Rocket RTL.

A.7 Replacing Spike's built-in PLIC / CLINT / Debug Module / Boot ROM with the SMC's TLM versions

Spike ships its own `plic_t`, `clint_t`, `debug_module_t`, and a tiny boot ROM. They are RISC-V-spec-compliant and pass `riscv-tests` / `riscv-arch-test` regressions, so the obvious question is "why not just keep them and save the implementation effort?" The short answer: at the **architectural register layer** Spike's models are equivalent, but at the **system layer** they are missing every SMC-specific concern. This sub-section first justifies the replacement, then explains how to do it (and finally how to keep Spike's built-ins as scaffolding during early bring-up).

A.7.1 Spike's built-ins vs. the SMC's TLM modules

Concern	Spike's <code>pllc_t</code> / <code>clint_t</code>	SMC requirement	Match?
RISC-V architectural register layout (<code>priority</code> , <code>enable</code> , <code>claim/complete</code> , <code>MSIP</code> , <code>MTIMECMP</code> , <code>MTIME</code>)	Canonical	Canonical	✓
Number of PLIC sources	Configurable; default ≈ 31 ; data structures scale but path is rarely exercised past ~ 256	332 sources (326 active)	⚠ Works in principle, off the well-trodden path
Number of PLIC contexts	Configurable (1/hart by default; 2 with S-mode)	2×4 cores = 8 contexts (M+S per hart)	✓
Address mapping into the system	Hard-coded in <code>sim_t::make_mems()</code> / <code>dts.cc</code> to "virt" board addresses; rebasable at construction only	SMC PLIC at BASE + 0x0400_0000 (4 MB), CLINT at BASE + 0x0800_0000 (64 KB), with <code>LOCAL_BASE</code> / <code>GLOBAL_BASE</code> aliasing rewritten at runtime	⚠ One-shot rebase OK; runtime <code>GLOBAL_BASE</code> updates not followed
Source-line wiring (which peripheral drives which source bit)	Spike has no concept of "which IP" — <code>set_interrupt_level(id, lvl)</code> only	Spec-defined IRQ-ID-to-peripheral map (e.g. PVT temp = #283, mailbox(0) = #N, BEU#k = #M, ...)	Equal effort either way
Reachability from external AXI masters (<code>sys_axi_in</code> , <code>sep_axi_in</code> , <code>jtag_axi_in</code>)	None — built-ins live <i>inside</i> Spike. External masters can only get there by funnelling TLM transactions back into <code>simif_t::mmio_store</code> , an out-of-process hack	BMC, SEP and JTAG2AXI must read/write PLIC and CLINT through the fabric and <code>axi_filters</code>	✗
Inbound security checks (<code>prot[2:0]</code> , <code>source_id</code> , secure / non-secure, M/S/U enforcement, filter ranges)	Not modelled	SMC enforces per-source filter rules and rejects mismatched <code>source_id</code> / <code>prot</code>	✗
Multi-clock-domain semantics	Single simulator time	CLINT on <code>clk_ref_i</code> , PLIC on <code>clk_smc_i</code> , peripheral IRQs cross from <code>clk_periph_i</code>	✗ Need to fake CDC at the boundary anyway
<code>mtime</code> distribution to OCTS / telemetry / log timestamps	<code>mtime</code> is internal to Spike's <code>clint_t</code>	OCTS and the log engine read <code>mtime</code> (and OCTS pulses) for system-wide timestamping; OCTS injects into CLINT in some modes	✗ Cross-IP visibility lost
WDT / BEU / Mailbox interrupt aggregation, NMI lines, threshold IRQs from PVT/AVS, FLR isolation gating PLIC outputs	None of these IPs exist in Spike	All exist as SMC TLM modules and connect into PLIC	✗ Must be wired externally regardless
SystemC visibility (VCD trace, <code>dump_state()</code> , back-door socket for the test bench, register-bit assertions)	None — pure C++ inside the ISS process	Required by §3 of <code>03_SMC_Test_Plan.md</code> (claim/complete races, threshold tests, multi-context starvation, ...)	✗
Coverage instrumentation (line / branch / register / IRQ-source coverage feeding <code>lcov</code>)	Not exposed	Required by §6 of the test plan	✗

Concern	Spike's <code>plic_t</code> / <code>clint_t</code>	SMC requirement	Match?
Backend portability (Spike → TGC → Dromajo → Rocket Verilator)	Reusing Spike's PLIC/CLINT ties the cluster to Spike; TGC and Rocket-Verilator have their own (subtly different) PLIC/CLINT models	Backend-agnostic by design	✗ Lock-in
Implementation effort	Zero	One SC_MODULE each, ~600 lines of C++ — both already specified in §4 and §5	Not free, but bounded

A worked example

Test from `03_SMC_Test_Plan.md`: "External BMC over `sys_axi_in` must be **denied** when it tries to claim a PLIC interrupt belonging to an M-mode-only context."

- With Spike's `plic_t`: the BMC's TLM `b_transport` arrives at the SMC fabric, the inbound `axi_filter` would *like* to check it against PLIC context rules — but the PLIC isn't a TLM target it can introspect, so the only options are (a) bake the filter rule into the fabric (couples two IPs that should be independent) or (b) forward the access into `simif_t::mmio_store` and lose the `prot` / `source_id` info because Spike's PLIC doesn't read them. Either way you've lost the ability to *test* what the spec says about access control.
- With our `plic` SC_MODULE: the access goes fabric → `axi_filter[k]` → `plic.reg_socket`; checks happen in the modules they belong to; the test bench inspects PLIC state via the back-door socket; and the result lands in coverage as both an `axi_filter` event and a `plic.claim_denied` event.

The same reasoning repeats for: external CLINT writes from JTAG, OCTS-driven `mtime` correction, WDT-triggered NMI gating, FLR isolation cutting off PLIC outputs to externally-visible IRQ lines, and ATB telemetry reading `mtime`.

Conclusion: for *correctness* alone Spike's built-ins are equivalent at the register layer; they are inadequate at the system layer, and verifying the SMC against `03_SMC_Test_Plan.md` requires the SystemC versions. They are kept available only as an early-bring-up scaffold (§A.7.3).

A.7.2 How to suppress Spike's built-ins (production mode)

Spike default	What ships with Spike	How to suppress	What the SMC uses instead
<code>clint_t</code>	<code>riscv/clint.cc</code>	Don't add it to <code>cfg_t::devices</code> and don't pass <code>--clint</code> / builtin-clint flags to <code>cfg_t</code> .	<code>clint</code> SC_MODULE (§5)
<code>plic_t</code>	<code>riscv/plic.cc</code>	Same — leave it out of the device map.	<code>plic</code> SC_MODULE (§4)
<code>debug_module_t</code>	<code>riscv/debug_module.cc</code>	Construct <code>processor_t</code> with a dummy <code>debug_module_config_t{0}</code> and don't register it as a device.	<code>debug_module</code> SC_MODULE (§13)
Built-in boot ROM	small array in <code>sim_t</code>	Don't use <code>sim_t</code> at all — drive <code>processor_t</code> directly.	<code>boot_rom</code> SC_MODULE (§8)
HTIF / <code>tohost</code>	<code>fesvr/htif.cc</code>	Optional. Keep ON for printf-style firmware bring-up; turn OFF for production tests.	(kept as-is, see §A.8)

Because we do *not* instantiate `sim_t`, none of those built-ins exist in the address map; every load/store from Spike to those addresses falls through `simif_t::mmio_load` / `mmio_store`, which the bridge forwards over TLM to our `plic` / `clint` / `debug_module` / `boot_rom` modules. Round trip is:

```

Spike CSR mip ← set_irqs() ← irq_aggregator() ← plic.ctx_out[hart] (sc_signal)
                                     ▲
Spike load/store to 0xC400_xxxx → simif::mmio_load/store → bridge.xfer() → plic.reg_socket

```


So Spike runs the cores, but **all RISC-V-architectural devices are owned by our SystemC modules**. This is what guarantees that bugs in the SMC's PLIC / CLINT / Debug behaviour are caught by the model rather than hidden by Spike's built-ins.

A.7.3 Early-bring-up scaffold mode (`use_spike_builtin_plic_clint = true`)

Before the SystemC `plic` and `clint` modules are functional, firmware engineers still need to boot something. The cluster therefore exposes a single boolean — defaulted to `false` — that re-enables Spike's built-in PLIC and CLINT and skips the binding of the corresponding `meip` / `seip` / `mtip` / `msip` ports. Everything else (`mailbox`, `boot_rom`, `scratchpad_sram`, `uart_16550`, ...) stays the same.

Equivalence with the upstream first-implementation milestone. This scaffold configuration is intentionally identical to the "smallest useful integration" recipe in `riscv-sim/docs/systemc-tlm2-wrapper.md` — a `sim_t` instance with Spike's built-in `plic_t` / `clint_t`, Spike-owned `mem_t` for DRAM, one `tlm_bridge_device_t` window for SMC peripherals, and external interrupts injected via `sim->get_intctrl()->set_interrupt_level(id, level)`. In the SMC repo this is the contents of §A.14, milestone-1. Promoting from scaffold to production is a *single boolean flip* (`use_spike_builtin_plic_clint = false`) plus binding the SystemC `plic` / `clint` IRQ ports.

```
smc_cpu_cluster(smc_module_name n,
                unsigned      num_harts,
                std::string    isa = "rv64gc_zicsr_zifencei",
                std::string    priv = "msu",
                bool use_spike_builtin_plic_clint = false); // <-- scaffold switch
```

Implementation outline:

```
smc_cpu_cluster::smc_cpu_cluster(smc_module_name n, unsigned nh,
                                std::string isa, std::string priv,
                                bool builtin)
: sc_module(n), use_builtin_(builtin)
{
    build_harts(nh, isa, priv);

    if (use_builtin_) {
        // EARLY BRING-UP: enable Spike's models at the SMC bases so firmware can boot
        // before the SystemC plic/clint modules exist or are wired in.
        pool->enable_builtin_clint(/*base=*/SMC_LOCAL_BASE + 0x0800'0000ULL);
        pool->enable_builtin_plic (/*base=*/SMC_LOCAL_BASE + 0x0400'0000ULL,
                                /*sources=*/332,
                                /*contexts=*/2 * nh);
        // The IRQ aggregator is left dormant; meip/seip/mtip/msip ports are
        // tied low and the SystemC plic/clint modules MUST NOT be bound.
    } else {
        // PRODUCTION: built-ins suppressed; smc_top binds:
        //   plic.ctx_out[2h+0] -> meip_in[h]
        //   plic.ctx_out[2h+1] -> seip_in[h]
        //   clint.mtip_out[h]  -> mtip_in[h]
        //   clint.msip_out[h]  -> msip_in[h]
        SC_METHOD(irq_aggregator);
        /* ... sensitivity list as in §A.6 ... */
    }
}
```

Rules of use:

- `true` is allowed only in development branches and in the `iss-only` regression tier that runs `riscv-tests` against bare Spike. Production CI must run with `false` and gate-keep on the `03_SMC_Test_Plan.md` interrupt suite.
- The two modes are **mutually exclusive**: you may not bind the SystemC `plic` / `clint` while `use_builtin=true`. The `smc_top` constructor enforces this with an `sc_assert` so a misconfigured testbench fails loudly.

- All other SMC IPs (mailbox, BEU, WDT, telemetry, ...) keep producing IRQ lines unchanged. In `use_builtin=true` mode those lines are simply not wired into Spike's PLIC, so firmware that depends on them won't function — the scaffold mode is suitable for "boot to first instruction" and basic CSR/memory tests, not for system-level scenarios.

This gives the project a clean migration path: start in scaffold mode for week-0 firmware bring-up, and switch the flag to `false` (the default) the moment the SystemC `plic` / `clint` are integration-ready. After that point Spike's built-ins are dead code in the production build.

A.8 Firmware loading — FESVR ELF + optional HTIF

`fesvr` (front-end server) is the part of Spike that knows how to parse RISC-V ELF binaries and write them into the simulator's address space. We keep it because rewriting an ELF loader would be wasted effort.

```
#include <fesvr/elfloader.h>

void smc_cpu_cluster::load_image(const std::string& elf_path, uint64_t base) {
    auto syms = load_elf(elf_path.c_str(), &mem_callbacks_);
    // mem_callbacks_ is a simple struct that forwards each (addr, byte) to
    // smc_simif::host_write(), which uses the DATA channel of the bridge.
    if (auto it = syms.find("tohost"); it != syms.end()) tohost_ = it->second;
    if (auto it = syms.find("fromhost"); it != syms.end()) fromhost_ = it->second;
}
```

Two things to know:

- The loader writes through the *same* TLM bridge as runtime accesses. That means inbound filter behaviour, address remap and ECC/init logic in scratchpad SRAM are all exercised when the firmware is loaded — exactly the behaviour of a real boot from external master (`sys_axi_in`) DMA-ing the image into SRAM.
- HTIF `tohost` / `fromhost` is optional. If `tohost_` is non-zero, the cluster polls it at the end of every quantum and prints to stdout. This gives `printk`-style debug for early firmware bring-up without having to model UART or the log engine.

A.9 Debug / JTAG — bridging Spike's Debug Module spec to our SystemC

`debug_module`

Spike implements the full RISC-V Debug Specification, but inside its own process. We keep that machinery turned off (§A.7) and instead let our `debug_module` (§13) drive the cluster:

- `debug_module.halt_req[h]` (sc_signal) → `debug_irq_in` → `iss_irq_state.debug = true` → `processor_t::halt_request = true`.
- On the next `step()` Spike enters the debug state automatically.
- `debug_module` issues abstract commands (`COMMAND.cmdtype = 0`) by:
 1. Writing `DATA0..15` into the cluster's `ctrl` socket (back-door).
 2. The cluster translates this into `read_csr` / `write_csr` / `read_gpr` / `write_gpr` calls on the addressed hart via `iss_hart`.
- Memory reads/writes from the debug module (`COMMAND.cmdtype = 2`) are issued as **TLM transactions on the cluster's `data` socket**, with `source_id = JTAG_ID` in the extension. They therefore traverse the inbound filters, exactly like a real JTAG-to-AXI bridge.

For full JTAG (rather than direct DMI), wire the SCC `jtag_dtm` model (or any OpenOCD remote-bitbang server) to `debug_module`'s JTAG ports; nothing in the cluster changes.

A.10 Multi-hart, time and quantum keepers

Each hart has its own `SC_THREAD` and its own `tlm_utils::tlm_quantumkeeper`. The pattern (with the WFI refinement adopted from the upstream wrapper guide):

```

void smc_cpu_cluster::hart_thread(unsigned id) {
    auto& qk = qk_[id];
    auto& hart = *harts_[id];
    qk.reset();
    while (true) {
        if (!enable_[id]) { wait(quantum_event_[id]); continue; }
        if (!rst_core_n_i.read()) { hart.reset(); wait(rst_core_n_i.posedge_event()); }

        // WFI policy (upstream wrapper guide). Skip the quantum entirely when the
        // hart is parked in WFI; only the IRQ aggregator (§A.6) wakes us back up.
        if (hart.is_waiting_for_interrupt()) {
            qk.sync(); // donate accumulated time first
            wait(quantum_event_[id]); // notified by irq_aggregator() on any IRQ
            hart.clear_wfi(); // arch-correct on next instruction; harmless if already cleared
            continue;
        }

        uint32_t retired = hart.step(quantum_insts_);

        qk.inc(sc_time(retired * cycle_period_, SC_NS));
        if (qk.need_sync()) qk.sync();
    }
}

```

A.10.1 Quantum defaults — reconciled with upstream

Two defaults coexist in the literature; the SMC picks the smaller one and explains why:

Source	<code>INSNS_PER_RTC_TICK</code>	<code>INTERLEAVE</code> (insns/quantum)	Quantum (@1 GHz)	Suitable for
Spike CLI / upstream wrapper guide	100	5000	5 μ s	OS boot / Linux regressions where IRQ latency is dominated by Linux scheduling
SMC default (<code>quantum_insts_ = 1000</code>)	100	1000	1 μs	Bare-metal SMC firmware where peripheral IRQ latency budgets (mailbox, OCTS, WDT) are sub- μ s
Lock-step / co-sim	1	1	1 ns	Commit-log diff against Rocket-Verilator (§A.12)

The `tlm_global_quantum` is kept aligned with `quantum_insts_ * cycle_period_` (1 μ s in the SMC default), and the per-IP rules in §0 — **Temporal Decoupling for Simulation Speed-Up** assume this 1 μ s global quantum. Switching to the upstream 5 μ s default is supported via `SMC_TD_MODE=os_boot` (mapped on top of the existing `SMC_TD_MODE=fast` / `lockstep` switches in §0.6) but requires re-running the bring-up regressions in §A.14 milestone 5 to confirm no IRQ-latency assertions trip.

Tuning knobs (unchanged; restated for completeness):

- `quantum_insts_` (default 1000): too small → SystemC scheduler thrashes; too big → IRQ latency rises.
- `cycle_period_`: derived from the modelled `clk_smc_i` frequency parameter; 1 ns by default.
- `tlm_global_quantum::instance().set(sc_time(1, SC_US))`: keep aligned with `quantum_insts_*cycle_period_`.

For lock-step / co-sim mode set `quantum_insts_ = 1` and `qk.set_global_quantum(SC_ZERO_TIME)`.

A.10.2 Interaction between WFI policy and the §0 sync rules

The WFI fast-path is consistent with the project-wide TD policy:

- Calling `qk.sync()` before `wait(quantum_event_[id])` discharges the cluster's accumulated local time before yielding — this is the **S2** rule from §0.3 ("sync before any `wait()` on an event whose firing depends on global time").

- `irq_aggregator()` is sensitive to every IRQ port and notifies `quantum_event_[id]` with `SC_ZERO_TIME` — this satisfies **S3** because the IRQ producer (PLIC/CLINT/BEU) has already emitted the IRQ at the correct global time, so the aggregator picks it up immediately.
- A hart that wakes from WFI does *not* need to clear `mip` itself; the aggregator's most recent `iss_irq_state` push (§A.6) already mirrors the live IRQ inputs.

A.11 Build system / CMake hooks

Layout:

```
external/
  spike/                # git submodule -> riscv-software-src/riscv-isa-sim
libsmc/
  cpu/
    iss_hart.h
    iss_bus_bridge.h
    iss_backend_spike.cpp # links libriscv.a
    iss_backend_tgc.cpp   # optional, links libtgc.a
    iss_backend_dromajo.cpp # optional, links libdromajo.a
    smc_cpu_cluster.h/.cpp
```

Top-level CMake snippet:

```
option(SMC_ISS_BACKEND "spike|tgc|dromajo" spike)

add_subdirectory(external/spike-cmake) # tiny shim that builds Spike via ExternalProject

add_library(smc_cpu_cluster STATIC
  cpu/smc_cpu_cluster.cpp
  cpu/iss_bus_bridge.cpp
  $<$<STREQUAL:${SMC_ISS_BACKEND},spike>:cpu/iss_backend_spike.cpp>
  $<$<STREQUAL:${SMC_ISS_BACKEND},tgc>:cpu/iss_backend_tgc.cpp>
  $<$<STREQUAL:${SMC_ISS_BACKEND},dromajo>:cpu/iss_backend_dromajo.cpp>)

target_link_libraries(smc_cpu_cluster
  PUBLIC SystemC::systemc
  PRIVATE
    $<$<STREQUAL:${SMC_ISS_BACKEND},spike>:spike::riscv spike::fesvr spike::softfloat spike::disasm>
    $<$<STREQUAL:${SMC_ISS_BACKEND},tgc>:tgc::iss>
    $<$<STREQUAL:${SMC_ISS_BACKEND},dromajo>:dromajo::core>)
```

The shim `external/spike-cmake/CMakeLists.txt` is ~30 lines and just runs Spike's autotools build inside `ExternalProject_Add`, then exposes imported targets `spike::riscv` etc. It also applies any local Spike patches the SMC depends on (§A.15) before running `configure`.

A.11.1 Alternative — Spike's upstream `--with-systemc` build path

The upstream Spike wrapper guide proposes adding the wrapper *into* the Spike tree as an MCPPBS sub-project, gated on a new `--with-systemc=<prefix>` configure argument. The build then produces `libspike-systemc.{a,so}` that links against `libriscv`, `libfesvr` and `libsystemc`, and the `spike` CLI executable is left unchanged.

The two integration paths are compared below:

Path	Where the wrapper source lives	Build system	Pros	Cons
CMake <code>ExternalProject</code> (SMC default, §A.11)	<code>libsmc/cpu/</code> in the SMC repo	CMake	One source tree per project; SMC-specific <code>iss_hart</code> and bridge stay close to the rest of the SMC code; Spike submodule is read-only and patch-applied at configure time	Wrapper is invisible to the upstream Spike community; updating Spike needs SMC-side rebuild
Spike <code>--with-systemc</code> (upstream-style, §A.11.1)	<code>riscv-isa-sim/systemc/</code> (a new sub-project of <code>MCPPBS_SUBPROJECTS</code>)	autotools	Wrapper ships <i>with</i> Spike; downstream consumers get it for free; CI in <code>riscv-isa-sim</code> validates it	Couples wrapper releases to Spike releases; SMC-specific <code>smc_axi_extension</code> would have to be parameterised or layered on top

SMC policy: keep the SMC-specific wrapper (`iss_hart` , `iss_bus_bridge` , `smc_simif` , `smc_cpu_cluster`) in the SMC repo, but contribute the upstream-generic primitives (`sim_t::start_embedding()`), a parameterised `tlm_bridge_device_t` , the `INSNS_PER_RTC_TICK` / `INTERLEAVE` defaults exposed via `cfg_t`) back to Spike via the `--with-systemc` path. Once those primitives land upstream, the SMC repo's local Spike patches (currently applied by `external/spike-cmake/`) shrink to zero.

A.11.2 Outstanding Spike-side fixes the wrapper depends on

The upstream wrapper guide flags one concrete TODO that the SMC integration needs:

- `external_sim_device_t::size()` returns `PGSIZE` with a `// TODO` in the current Spike sources. If the SMC ever adopts the `tlm_bridge_device_t` flavour of §A.4.1 instead of the `simif_t::mmio_*` override, the bridge device must report the *full* mapped region size, not one page. The fix is one-line and is shipped as part of the SMC's `external/spike-cmake/` patch set; it is also the natural first patch to upstream as part of §A.11.1.

A.12 Co-simulation with Dromajo / Rocket Verilator (commit-log diff)

When Spike is built with `--enable-commitlog` , every retired instruction emits a line of the form

```
core 0: 3 0x0000000080000004 (0x00000513) x10 0x0000000000000000
```

Dromajo emits the same format. Rocket-Verilator (when `+verbose` is set) emits a compatible `Cn ...` trace. For sign-off the SMC test bench:

1. Runs the same firmware under `iss_backend_spike` and against the Rocket-Verilator model in lock-step (one instruction per quantum).
2. Captures both commit logs.
3. Runs `tools/diff_commits.py` (CHIPS-Alliance script) to flag any divergence — typically CSR mismatches due to features Rocket implements that Spike doesn't (or vice-versa).

Because both sims drive the same SystemC fabric, **the only diff is in the cluster** — exactly what we want to validate.

A.13 Worked example — booting SMC firmware under the wrapper

```
int sc_main(int, char**) {
    smc_top dut("dut");
    driver drv("drv");

    drv.sys_axi.bind(dut.sys_axi_in);
    drv.sep_axi.bind(dut.sep_axi_in);
    drv.jtag_axi.bind(dut.jtag_axi_in);
    dut.output_axi.bind(drv.noc);

    // 1. Power-on / cold reset.
    drv.assert_powergood();
    drv.assert_cold_reset();
    sc_start(100, SC_NS);
    drv.deassert_cold_reset();

    // 2. Load firmware ELF into Boot ROM via JTAG (or directly via DMI).
    dut.cpu.load_image("smc_fw.elf", 0xC0040000);

    // 3. Release the Rocket cores.
    dut.cpu.write_ctrl(/*offset*/0x040, /*CORE_ENABLE*/0xF);

    // 4. Run until firmware writes the OCCP "ready" mailbox, or 10 ms.
    sc_start(10, SC_MS);

    // 5. Cross-check.
    EXPECT_EQ(dut.mb.channel(0).status(), MAILBOX_OCCP_READY);
    return 0;
}
```

Because the wrapper routes every Spike load/store through TLM, this single test exercises:

- `boot_rom` reads (DMI fast path),
- `scratchpad_sram` initialisation (`init_done_o` gating),
- `clint::mtip` / `msip` driving Spike CSRs through the aggregator,
- `plic` claim/complete via Spike's CSR access through `simif_t::mmio_load/store`,
- `mailbox_unit` write path,
- inbound/outbound filter `prot` / `source_id` checks (since the bridge sets the extension correctly).

If any of those break, the firmware will hang or trap and the test bench will fail the deadline — without any new test code beyond loading the ELF and pulling reset.

A.14 First-implementation milestone (mapped from the upstream wrapper guide)

The Spike wrapper guide proposes a six-step "smallest useful integration" that gets a bare-metal ELF running through the SystemC scheduler before the SMC's own PLIC/CLINT/peripherals are wired up. The SMC adopts that recipe verbatim as **milestone 1** of the cluster bring-up plan; milestones 2–4 promote the model from scaffold to production. Each step below points at the appendix sub-section that holds the implementation detail.

#	Upstream-recipe step	SMC realisation	Where
M1.1	Add <code>sim_t::start_embedding()</code> and <code>sim_t::step_embedding(size_t)</code> (and the matching <code>done_embedding()</code> / <code>exit_code_embedding()</code>)	Carried as the <code>0001-add-embedding-api.patch</code> applied to the pinned Spike submodule	§A.1, §A.11
M1.2	Build an external SystemC module (<code>spike_tlm</code> upstream-style) that links against Spike	<code>smc_cpu_cluster</code> with <code>SMC_ISS_BACKEND=spike</code> is exactly that module — no separate <code>spike_tlm</code> source file	§A.11
M1.3	Use Spike-owned <code>mem_t</code> for DRAM	Boot ROM and Scratchpad SRAM are mode-A by default	§A.5.1
M1.4	Add one <code>tlm_bridge_device_t</code> window for SMC peripherals	Either <code>tlm_bridge_device_t(0xC000_0000, 0x4000_0000, mmio_socket, &local_delay)</code> or the equivalent <code>simif_t::mmio_*</code> override (SMC default)	§A.4, §A.4.1
M1.5	Route external interrupts through Spike's existing PLIC	<code>use_spike_builtin_plic_clint = true</code> ⇒ <code>sim->get_intctrl()->set_interrupt_level(id, level)</code> for every SMC IRQ source	§A.7.3
M1.6	Run a bare-metal ELF from the SMC simulation and exit through <code>tohost</code>	<code>dut.cpu.load_image("smc_fw.elf", 0xC0040000);</code> <code>sc_start(10, SC_MS);</code>	§A.13

After M1.6 is green, milestones 2–4 promote the model:

#	Step	Required change	Where
M2	Replace Spike's PLIC and CLINT with the SMC TLM versions	Flip <code>use_spike_builtin_plic_clint = false</code> ; bind <code>plic.ctx_out[]</code> / <code>clint.{mtip,msip}_out[]</code> to <code>meip_in[]</code> / <code>seip_in[]</code> / <code>mtip_in[]</code> / <code>msip_in[]</code>	§A.6, §A.7.2
M3	Replace Spike's Debug Module with the SMC <code>debug_module</code>	Stop registering Spike's <code>debug_module_t</code> ; bind <code>debug_module.halt_req[]</code> to <code>debug_irq_in</code>	§A.9
M4	Enable temporal-decoupling fast mode and the WFI fast-path	<code>SMC_TD_MODE=fast</code> , set <code>quantum_insts_ = 1000</code> (default), confirm WFI policy in §A.10 is active	§A.10, §0.6

Each milestone is paired with a corresponding test class in `03_SMC_Test_Plan.md`:

Milestone	Test classes that gate promotion
M1	<code>RV-ISA-SANITY</code> , <code>BARE-METAL-MMIO</code> , <code>PLIC-IRQ-SCAFFOLD</code> (drives via Spike's <code>set_interrupt_level</code>)
M2	<code>PLIC-CLAIM-COMPLETE</code> , <code>PLIC-MULTI-CONTEXT</code> , <code>CLINT-MTIME-COHERENCE</code> , <code>BMC-CLAIM-FILTER</code>
M3	<code>DM-ABSTRACT-CMD</code> , <code>DM-MEMORY-ACCESS-VIA-FABRIC</code> , <code>JTAG-HALT-RESUME</code>
M4	<code>WFI-WAKEUP-LATENCY</code> , <code>OCTS-TICK-DRIFT</code> , <code>IRQ-LATENCY-BUDGET</code>

This staging gives the firmware team a usable Spike-driven model on day one (M1) and converts it into the system-level signoff model only as each SystemC peripheral becomes integration-ready.

A.15 Risks and required Spike upstream changes

The upstream wrapper guide is explicit that Spike's principal *public* API is the RISC-V ISA itself; its internal C++ classes (`sim_t`, `processor_t`, `bus_t`, `htif_t`, ...) carry no compatibility guarantee and are reshaped freely between releases. The SMC-side mitigations are:

Risk	Mitigation in the SMC repo
Spike internal C++ API breakage between revisions	Spike is consumed as a git submodule pinned to a specific commit SHA (the <code>external/spike</code> submodule's <code>HEAD</code> is recorded in the SMC repo's lockfile). Bumping the SHA is an explicit MR that re-runs the M1-M4 regressions of §A.14.
Patch drift while the embedding API and <code>tlm_bridge_device_t</code> size fix are not yet upstream	All local patches live under <code>external/spike-cmake/patches/</code> and are applied by <code>ExternalProject_Add(... PATCH_COMMAND git am ...)</code> . Each patch carries a header line <code>Upstream-Status: <Pending PR # / Submitted / Merged></code> . Once a patch is merged upstream, the next Spike SHA bump removes the local copy.
Blocking <code>htif_t::run()</code> accidentally re-introduced through a <code>sim_t::run()</code> call somewhere in the wrapper	Static-analysis lint rule (<code>tools/lint/no_spike_blocking.cmake</code>) refuses the build if any SMC source file references <code>sim_t::run</code> , <code>htif_t::run</code> , or <code>sim_t::interactive</code> .
External masters mutating Spike memory while a hart is mid-step	All inbound TLM transactions on Spike-owned memory ranges are serialised through the cluster's <code>data_socket</code> target callback, which delays the access until the hart's <code>SC_THREAD</code> yields (i.e. between <code>step()</code> calls). The <code>iss_bus_bridge</code> is documented as single-threaded; no peripheral may call into Spike directly from another <code>SC_THREAD</code> .
MMIO width assumptions differing between Spike's MMIO helper macros and the SMC fabric	Enforced by the access-width discipline of §A.4.2; unit-tested in <code>BARE-METAL-MMIO</code> (byte/half/word/dword sweep).
Big-endian configurations silently dropping byte-lane swaps	Big-endian builds are off by default. If ever turned on (<code>--enable-dual-endian</code> + <code>--big-endian</code>), the M1 regressions must include explicit byte-lane tests on every MMIO window.
Multi-hart <code>mtime</code> skew when using Spike's built-in CLINT in scaffold mode	Documented limitation of M1; OCTS-driven <code>mtime</code> correction (which requires the SMC TLM CLINT) only works from M2 onwards.

Required or recommended upstream contributions (in priority order):

1. `sim_t::start_embedding()`, `step_embedding(n)`, `done_embedding()`, `exit_code_embedding()`. One-time, additive. Already proposed in `riscv-isa-sim/docs/systemc-tlm2-wrapper.md`.
2. **Configurable-size TLM bridge device** — either replace `external_sim_device_t::size()`'s `PGSIZE` placeholder with a constructor-injected size, or introduce a fresh `tlm_bridge_device_t` class as the upstream guide proposes.
3. **MMIO forwarding and external-IRQ injection unit tests** in `riscv-isa-sim/ci-tests/` to lock down the embedding API surface against future regressions.
4. **Optional non-blocking HTIF service method** (`htif_t::service_step()`) so that `tohost` / `fromhost` polling can be interleaved more tightly than one full instruction quantum.
5. **Optional public local-IRQ / timer API** so that an external CLINT (the SMC TLM module) can drive `mip.MTIP` / `mip.MSIP` without backdoor CSR writes.

These five items, once upstream, would let the SMC's local Spike patch-set go to zero — at which point the SMC can pin a stock release tag of `riscv-isa-sim` and the entire integration becomes a thin layer in the SMC repo plus a documented `--with-systemc` build flag in Spike.